



Stock Price Prediction using Algorithmic Trading

A Comprehensive Literature Review

Submitted by

Anik Das
Agnirudra Banerjee
Mohar Mukherjee
Bibhas Roy

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Sister Nivedita University
Newtown, Kolkata, West Bengal

2025



Stock Price Prediction using Algorithmic Trading

Project Submitted in Partial Fulfillment of the Requirements for the Degree of
BACHELOR OF TECHNOLOGY (CSE)

Name	Enrollment	Email ID
Agnirudra Banerjee	2211200010029	agnirudrabanerjee777@gmail.com
Bibhas Roy	2211200010001	abc.3gbibhas@gmail.com
Mohar Mukherjee	2211200010010	MOHARMUKHERJEE2004@gmail.com
Anik Das	2211200010038	anik200365@gmail.com

Submission Date: 21-November, 2025

Under the supervision of

Dr. Soma Datta

Assistant Professor

Department of Computer Science

Sister Nivedita University, Newtown

Kolkata, West Bengal

Sister Nivedita University

Newtown, Kolkata, West Bengal

November, 2025

DECLARATION

We hereby declare that the project work entitled “**Stock Price Prediction using Algorithmic Trading**” submitted to the Department of Computer Science, Sister Nivedita University, is a record of an original work done by us under the supervision of **Dr. Soma Datta**, Assistant Professor, Department of Computer Science, Sister Nivedita University.

This project work is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

.....
Agnirudra Banerjee

.....
Bibhas Roy

.....
Mohar Mukherjee

.....
Anik Das

Place: Kolkata

Date: November, 2025

CERTIFICATE

This is to certify that the project report entitled “**Stock Price Prediction using Algorithmic Trading**” submitted by:

Agnirudra Banerjee	(ID: 2211200010029)
Bibhas Roy	(ID: 2211200010001)
Mohar Mukherjee	(ID: 2211200010010)
Anik Das	(ID: 2211200010038)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of Sister Nivedita University, is a bona fide record of the work carried out under my supervision and guidance.

Dr. Soma Datta

Assistant Professor

Department of Computer Science

Sister Nivedita University, Newtown, Kolkata

Contents

1	Introduction	1
1.1	Context of the Study	1
1.2	Objectives	2
1.3	Scope and Organization	2
1.4	Contributions	2
2	Literature Review	4
2.1	Introduction to Literature Survey	4
2.2	Historical Approaches	4
2.2.1	Evolution of Stock Prediction Methods	4
2.2.2	From Early Statistical Tools to Advanced Time-Series Models	4
2.2.3	From ARIMA/GARCH to Machine Learning Models	5
2.2.4	From Machine Learning to Deep Learning	5
2.2.5	From Deep Learning to Modern Transformer and Advanced Models	5
2.3	Existing Systems / Related Work	6
2.3.1	Traditional Statistical Methods	6
2.3.2	Machine Learning Approaches	7
2.3.3	Deep Learning-Based Approaches	7
2.3.4	Advanced Neural Network Architectures	8
2.3.5	Sentiment-Based Methods	9
2.4	Hybrid Architectures	10
2.4.1	ARIMA-LSTM Hybrid Models	10
2.4.2	CNN-LSTM Hybrid Models	10
2.4.3	Attention-Enhanced Hybrid Models	11
2.4.4	Multimodal Hybrid Models	12
2.4.5	Feature-Level Hybridization	12
2.4.6	Price + Sentiment Hybrid Models	12
2.5	Comparative Analysis of Hybrid Models	13
2.5.1	Why Hybrid Models Represent the Future	13
2.6	Limitations of Existing Systems	15
2.6.1	Statistical Model Constraints	15
2.6.2	Machine Learning Limitations	15
2.6.3	Feature Representation Challenges	16
2.6.4	Computational and Real-Time Constraints	16
2.6.5	Overfitting and Generalization Issues	16
2.6.6	Sentiment Analysis and Integration Gaps	16
2.7	Comparative Study / Gap Analysis	17
2.7.1	Method Comparison: Statistical vs. Machine Learning vs. Deep Learning	17
2.7.2	Performance Comparison from Literature	18
2.7.3	Integration Gaps and Research Deficiencies	18
2.8	Summary of Literature Review	20

2.8.1	Key Findings	20
2.8.2	Conclusion	22
3	Problem Identification	24
3.1	Problem Statement	24
3.2	Context and Background of the Problem	24
3.3	Need and Importance of Solving the Problem	25
3.3.1	Economic Impact	25
3.3.2	Scientific Advancement	25
3.3.3	Market Efficiency	25
3.3.4	Risk Management	25
3.4	Scope and Project Objectives	26
3.4.1	Scope	26
3.4.2	Objectives	26
3.5	Anticipated Challenges	27
3.5.1	Data Quality and Availability	27
3.5.2	Non-stationarity and Regime Changes	27
3.5.3	Overfitting vs. Generalization	27
3.5.4	Computational Resources	27
3.5.5	Evaluation Metrics	28
3.5.6	Interpretability and Trust	28
3.5.7	Benchmark Comparison	28
3.5.8	Sentiment Data Challenges	28
4	Problem Methodology	30
4.1	System Architecture Overview	30
4.2	Data Collection and Preprocessing	30
4.2.1	Data Source and Characteristics	30
4.2.2	Data Cleaning	32
4.2.3	News Data Collection	33
4.2.4	Train-Test Split	33
4.3	Feature Engineering	33
4.3.1	Technical Indicators	34
4.3.2	Temporal Features	35
4.3.3	Lag Features	35
4.3.4	Feature Selection and Dimensionality	35
4.3.5	Sentiment Analysis Pipeline	36
4.3.6	Sentiment Feature Engineering	40
4.4	Model Development	41
4.4.1	SARIMA Model	41
4.4.2	Prophet Model	42
4.4.3	XGBoost Model	43
4.4.4	BiLSTM + Attention Model	44

4.4.5	Hybrid SARIMA-XGBoost Model	47
4.5	Evaluation Metrics	50
4.5.1	Regression Metrics	51
4.5.2	Directional Accuracy	51
4.5.3	Trading Performance Metrics	51
4.5.4	Model Comparison	52
4.6	Implementation Details	52
4.6.1	Development Environment	52
4.6.2	Computational Resources	52
4.6.3	Reproducibility	52
4.7	Summary	52
5	Results and Analysis	54
5.1	Experimental Setup	54
5.2	Exploratory Data Analysis	54
5.3	Model Performance Evaluation	55
5.3.1	Statistical Models	55
5.3.2	Machine Learning Models	55
5.3.3	Deep Learning Models	56
5.4	Hybrid Model: SARIMA + XGBoost	56
5.4.1	Theoretical Foundation and Methodology	56
5.4.2	Implementation Configuration	57
5.4.3	Performance Results	58
5.5	Sentiment-Augmented Model Results	58
5.5.1	Expected Performance	59
5.5.2	Ablation Study Design	59
5.5.3	Feature Importance Analysis	60
5.5.4	Walk-Forward Validation	60
5.6	Comprehensive Comparison	60
5.7	Discussion	61
5.7.1	Key Findings and Implications	61
5.7.2	Practical Implications	63
5.7.3	Limitations and Caveats	64
5.7.4	Literature Alignment	65
5.7.5	Future Research Directions	65
5.8	Conclusion	66

List of Figures

1	Complete System Architecture for Sentiment-Augmented Hybrid Stock Prediction	31
2	Sentiment Analysis Pipeline: End-to-End Data Flow from Raw News to XG-Boost Integration	36
3	BiLSTM with Attention Mechanism Architecture	45
4	Hybrid SARIMA-XGBoost Workflow	48
5	Historical Closing Price of AAPL (2010-2025)	54
6	Feature Correlation Heatmap	55
7	BiLSTM + Attention Training History	57
8	Hybrid Model (SARIMA + XGBoost) Predictions vs Actual Prices	58
9	Comparison of All Model Predictions	62

List of Tables

1	Historical Evolution of Stock Prediction Methods	4
2	Comprehensive Comparison of Hybrid Models	14
3	Comparative Analysis of Stock Prediction Approaches	18
4	Formal Summary of Identified Research Gaps in Stock Prediction and Sentiment Analysis	21
5	Sentiment-Augmented Model Performance Targets vs. Baseline Hybrid	59
6	Comprehensive Performance Comparison of All Models	61

ACKNOWLEDGEMENT

We would like to express our deep sense of gratitude and profound respect to our supervisor, **Dr. Soma Datta**, Assistant Professor, Department of Computer Science, Sister Nivedita University, for her valuable guidance, constant encouragement, and constructive criticism throughout the course of this project work. Her expertise and insight have been invaluable in shaping our understanding of the subject.

We are also grateful to the **Department of Computer Science** and **Sister Nivedita University** for providing us with the necessary facilities and an environment conducive to learning and research.

Finally, we would like to thank our families and friends for their unwavering support and encouragement during this endeavor.

.....
Agnirudra Banerjee

.....
Bibhas Roy

.....
Mohar Mukherjee

.....
Anik Das

ABSTRACT

The prediction of stock market prices is a challenging task due to the non-linear, volatile, and sentiment-driven nature of financial time series data. Traditional statistical models like ARIMA have limitations in capturing non-linear patterns, while deep learning models like LSTM excel at sequential data but may struggle with noise. Price-only models, regardless of architectural sophistication, are structurally blind to exogenous information shocks—such as earnings surprises, macroeconomic announcements, and shifts in public sentiment—until those shocks are already reflected in historical prices. This literature review comprehensively explores the evolution of stock prediction methodologies, with a specific focus on **Hybrid Models** that combine the strengths of multiple approaches (e.g., ARIMA-LSTM, CNN-LSTM), and extends the analysis to **Sentiment-Aware Hybrid Architectures** that integrate financial news sentiment into the prediction pipeline.

We analyze various hybrid architectures, comparing their mechanisms, strengths, and limitations. The review highlights how hybrid models consistently outperform standalone models by effectively handling both linear and non-linear components, as well as incorporating diverse data sources such as technical indicators and macroeconomic variables. Building on this foundation, we propose a sentiment-augmented extension of the SARIMA-XGBoost hybrid model that integrates a domain-adapted sentiment analysis pipeline featuring dual-head sarcasm correction, principled temporal alignment of news publication timestamps to trading days, and confidence-weighted sentiment aggregation. The system addresses critical gaps identified in modern sentiment analysis literature—including sarcasm and irony detection, tokenization drift in financial language, domain sensitivity, and uncertainty quantification—and feeds structured sentiment features into the XGBoost residual learning stage, enabling the model to capture linear trends, non-linear residual patterns, and sentiment-driven price dynamics within a unified three-component decomposition $y_t = L_t + N_t + E_t + \epsilon_t$. Furthermore, this document identifies research gaps in current methodologies, particularly regarding multimodal data integration, the lack of adaptive learning mechanisms for regime shifts, and the absence of robust risk-aware prediction frameworks. These identified gaps and the proposed sentiment-aware architecture lay the foundation for more robust and accurate stock prediction systems.

1 Introduction

1.1 Context of the Study

The stock market is a part of the financial system. The stock market shows how the economy is doing and moves money to where it's needed. I find that predicting the stock market is hard. The stock market moves in complex ways, has swings, and does not follow a straight line. The stock market data is a time series that can change quickly. The stock prices move fast because they stock prices are affected by things. The stock prices react to how a company performs, to numbers to world events, to market mood and to investor behavior.

Traditional statistical models have been used for decades to forecast stock prices. Traditional statistical models often have trouble with market shifts, regime changes and non-linear patterns in the data. The rise of machine learning and deep learning gives researchers tools to study the patterns. Machine learning and deep learning can handle patterns that traditional statistical models miss. I have seen that modern computational techniques can learn from large amounts of data. Modern computational techniques can find relationships that're hard to see with analysis or with classical statistical methods. When modern computational techniques look at years of data modern computational techniques can spot connections that no one would notice by just looking at the numbers or by using traditional statistical models.

However, even the most sophisticated price-based models—whether statistical, machine learning, or deep learning—share a fundamental structural limitation: they operate entirely on historical OHLCV (Open, High, Low, Close, Volume) features and derived technical indicators. These models have no mechanism to respond to sudden exogenous information shocks, such as a negative earnings surprise, a CEO resignation, or a macroeconomic policy change, until those shocks are already reflected in the price history. Financial markets are fundamentally information-processing systems where news, earnings announcements, analyst opinions, and public sentiment all contribute to price discovery. A model that can extract and quantify the sentiment embedded in financial news, and integrate it into its prediction pipeline, effectively gains advance information that price-only models lack.

Integrating sentiment analysis into financial prediction is not straightforward. Modern sentiment models suffer from several well-documented gaps: financial language contains domain-specific jargon, hedged expressions, and sarcasm that general-purpose sentiment models misinterpret; news publication timestamps do not directly align with trading day boundaries, creating temporal mismatch problems; most sentiment pipelines produce deterministic point estimates with no uncertainty quantification; and models trained on general corpora experience tokenization drift when applied to financial text. This work is therefore motivated by the convergence of two research problems: the need for sentiment-aware trading models, and the need for robust, domain-aware sentiment analysis that addresses the gaps documented in existing literature.

1.2 Objectives

This literature review aims to:

- Examine the evolution of stock prediction methodologies from traditional statistical approaches to modern hybrid deep learning models.
- Analyze the specific contributions of ARIMA, LSTM, BiLSTM, and hybrid architectures in the context of financial forecasting.
- Identify the limitations and research gaps in existing systems, with particular attention to gaps in modern sentiment analysis methods including sarcasm detection, tokenization drift, domain sensitivity, temporal alignment, and uncertainty quantification.
- Establish the foundation for developing an improved hybrid prediction model.
- Design and implement a domain-adapted financial sentiment analysis pipeline that addresses the identified gaps, and integrate confidence-weighted, sarcasm-corrected sentiment features into the existing SARIMA-XGBoost hybrid prediction framework.
- Develop a principled mathematical formulation for sentiment scoring and its incorporation into the residual learning stage of the hybrid model.
- Evaluate the sentiment-augmented model against the baseline hybrid model and analyze the contribution of individual sentiment features through feature importance and ablation studies.

1.3 Scope and Organization

The review consists of three distinct sections, which follow each other in order. The second chapter of this review provides an extensive review of traditional methods and deep learning approaches, and advanced hybrid architectures, including sentiment-based methods and their integration into trading systems. The third chapter identifies essential problems and missing information that prove the need for improved hybrid forecasting systems to forecast stock market performance. The fourth chapter presents the complete methodology, including the sentiment analysis pipeline and its integration into the hybrid model. The fifth chapter presents the results, ablation studies, and a comprehensive discussion of findings and future directions.

1.4 Contributions

The contributions of this work are summarized as follows:

1. **Sentiment-Aware Hybrid Architecture.** We propose a novel three-stage prediction architecture that combines SARIMA for linear trend extraction, domain-adapted sentiment feature engineering, and XGBoost for joint non-linear residual and sentiment modeling. The architecture extends the standard two-component decomposition $y_t = L_t + N_t + \epsilon_t$

to a three-component decomposition $y_t = L_t + N_t + E_t + \epsilon_t$, where E_t represents the sentiment-driven component of price dynamics. To the best of our knowledge, this is one of the few works to explicitly address the temporal alignment problem in sentiment-trading integration within a hybrid statistical-ML framework.

2. **Dual-Head Sarcasm-Corrected Sentiment Scoring.** We introduce a sarcasm-aware sentiment scoring function that combines a primary FinBERT sentiment head with a secondary sarcasm detection head sharing the same transformer backbone. The final sentiment score $s_i^{\text{final}} = (1 - 2\lambda_i) \cdot s_i^{\text{raw}}$ is a confidence-weighted mixture that corrects for sarcastic financial commentary, which standard sentiment models incorrectly classify. This directly addresses one of the most significant gaps documented in modern sentiment analysis literature.
3. **Principled Temporal Alignment.** We define a formal temporal assignment function $\phi(\tau_i) = \min\{t \in \mathcal{T} : \text{MarketOpen}(t) > \tau_i\}$ that maps news publication timestamps to the correct trading day, accounting for after-hours publication, weekends, and market holiday effects. This ensures sentiment signals are assigned to the trading day on which they are first actionable, preventing information leakage in the training set.
4. **Uncertainty-Quantified Sentiment Features.** Rather than using raw sentiment scores as features, we derive an 8-dimensional structured sentiment feature vector $\mathbf{S}_t = [S_t, \Delta S_t, \sigma_{S,t}, N_t, D_t, S_{t-1}, S_{t-2}, S_{t-3}]$ comprising confidence-weighted aggregated sentiment, sentiment momentum, rolling sentiment volatility, normalized news volume, sentiment-price divergence, and lagged sentiment values. This provides the downstream XGBoost model with richer and more reliable sentiment signals than raw point estimates.
5. **Empirical Gap Analysis Validation.** We empirically validate the gaps identified in existing sentiment literature—particularly around domain sensitivity and tokenization drift—by comparing FinBERT against a general-purpose BERT model on financial news classification and demonstrating measurable performance differences. The ablation study design isolates the contribution of individual sentiment components, providing evidence-based guidance for future sentiment-trading integration work.

2 Literature Review

2.1 Introduction to Literature Survey

Stock price prediction has evolved through technological progress during the last fifty years because computational intelligence combined with financial market analysis. The combination of statistical modeling with machine learning algorithms and deep learning architectures allows developers to create advanced forecasting systems at unprecedented levels. The traditional methods based on linear assumptions and stationary conditions failed to detect the intricate financial market patterns. The current data-driven learning system detects patterns automatically through its own processes without needing human-made feature development.

The review investigates current stock price prediction methods which include statistical approaches and machine learning methods and deep learning systems and hybrid prediction systems. In addition, the review covers sentiment analysis techniques—from lexicon-based methods to domain-adapted transformers such as FinBERT—and examines how sentiment signals have been integrated into trading systems, along with the critical gaps that remain in this integration. The research follows the development of stock price prediction from initial random walk theories to present-day transformer-based, sentiment-aware, and multimodal prediction systems.

2.2 Historical Approaches

Stock prediction methods have undergone substantial development since the 1960s through successive periods which brought fundamental changes to modeling strategies.

2.2.1 Evolution of Stock Prediction Methods

Table 1: Historical Evolution of Stock Prediction Methods		
Era	Dominant Method	Key Improvement
1960s–1980s	Random walk, moving averages	Basic trend and noise understanding
1990s–2000s	ARIMA, GARCH	Strong statistical forecasting
2010–2015	ML models (SVM, RF)	Non-linear learning
2015–2020	LSTM, GRU, CNN	Long-sequence learning
2020–Present	Transformers, GNN, RL	Context-aware, long-range, multimodal prediction

2.2.2 From Early Statistical Tools to Advanced Time-Series Models

The first stage of stock prediction needed me to use fundamental concepts which included random walk theory and basic moving average calculations. The trading methods helped traders

find market directions yet they did not work for detecting intricate market patterns and unanticipated price fluctuations. The efficient market hypothesis ruled this period by stating that stock prices behave randomly and investors lack ability to forecast market movements.

The following period brought better results through the implementation of ARIMA and GARCH models which served as structured mathematical frameworks. The new methods delivered better statistical power which allowed researchers to study market patterns and seasonal market behavior and volatility changes through systematic approaches that earlier tools failed to provide. The market transition from random behavior to statistical pattern detection occurred during this period.

2.2.3 From ARIMA/GARCH to Machine Learning Models

The models ARIMA and GARCH proved useful but they required data to follow linear patterns. Real market behavior shows complex non-linear patterns because multiple variables create intricate relationships which linear models fail to detect.

Machine learning models advanced the field by learning non-linear data relationships directly from the input information. The prediction models SVM and Random Forest and boosting algorithms detected multiple feature relationships between price and volume and technical indicators. The models achieved superior prediction results than statistical methods because they identified complex patterns between features through automated processes that did not need predefined mathematical models.

2.2.4 From Machine Learning to Deep Learning

Machine learning models were effective, and their main limitation was that it was ineffective in the comprehension of sequences across long periods of time. However, stock prices are determined by trends that are carried out over days, weeks or even months. ML algorithms in the past kept the time points independent, necessitating the need to manually feature engineer time-varying variables.

This was enhanced with deep learning through sequence-based models, such as RNNs, LSTMs, and GRUs. These models were able to store long term dependencies, and to learn directly using raw price sequences without extensive feature engineering. The CNN-based time-series models introduced the capability of identifying short-term local trends like volatility clusters and momentum shifts. They combined automatic feature learning with time-varying modeling together, which formed a more robust base than the classical methods of machine learning.

2.2.5 From Deep Learning to Modern Transformer and Advanced Models

LSTMs and RNNs were also not efficient with very long sequences, slow to train, and information decayed over time because of the sequential nature of the processing.

The next major improvement was introduced with the help of transformers that made use of attention mechanisms. Transformers do not read data sequentially and instead consider all the time points at the same time and determine which ones are the most important. This enables

them to work with long sequences more efficiently, parallelize more quickly, and offer a more interpretable representation with attention weight visualization.

Graph Neural Networks (GNNs) made another step forward and attempted to model the effects of stocks on each other- industry correlations, supply chain relationships and market contagion effect, previously absent in other models. Reinforcement learning took the field a step further by changing the focus of simple forecasting to complete decision making, trading strategies optimized not only by predicting prices.

Lastly, the latest icyte of price data with news, social media, and market sentiment is called multimodal models, which enables the system to learn on several kinds of information rather than historical price. This holistic theory appreciates that markets are motivated by the flow of information over various channels.

2.3 Existing Systems / Related Work

2.3.1 Traditional Statistical Methods

The earliest methods of stock price prediction were based mainly on statistical time-series model. These approaches have formed the basis of time-dependent dependence and stochastic processes in financial data.

ARIMA Models: The AutoRegressive Integrated Moving Average (ARIMA) model has been a staple in time-series forecasting since the 1970s. The model is valued for its ability to capture linear trends, seasonality, and autocorrelation structures in data. As demonstrated by [1] in retail analytics, ARIMA effectively predicts consumer demand and optimizes inventory management. The model is mathematically expressed as:

$$\phi(B)(1 - B)^d y_t = \theta(B)\epsilon_t \quad (1)$$

where B represents the backward shift operator, d denotes the degree of differencing required to achieve stationarity, $\phi(B)$ represents the autoregressive polynomial, and $\theta(B)$ represents the moving average polynomial [1].

The study by [1] demonstrates ARIMA’s capability in identifying “temporal shopping behaviors” and “restocking trends” after weekends. In the context of stock prediction, this translates to detecting cyclic trading patterns, mean-reversion tendencies, and seasonal effects in stock prices. The model’s strength lies in its interpretability and solid theoretical foundation rooted in econometric theory.

However, ARIMA’s fundamental limitation is its linear assumption. As noted in [1], the model requires careful “differencing” to handle non-stationary data—a characteristic shared by volatile stock indices. When financial markets experience regime changes, structural breaks, or extreme volatility, ARIMA’s linear framework becomes inadequate. The model cannot capture non-linear interactions between features, sudden jumps in prices, or complex dependencies that characterize modern financial markets.

GARCH Models: Generalized AutoRegressive Conditional Heteroskedasticity (GARCH) models are an addition to ARIMA that deal with volatility clustering, i.e. the period of high

volatility is succeeded by a period of high volatility, and the period of low volatility is followed by a period of low volatility. Although the ARIMA is a model of the conditional mean of the series, GARCH is a model of the conditional variance as it is especially applied in risk management and option pricing. But, similar to ARIMA, GARCH has parametric assumptions and is poor at extreme market regimes.

2.3.2 Machine Learning Approaches

Machine learning brought a big change in comparison to the traditional statistics algorithms with this feature of non-linear modelling with no rigid distributional assumptions.

Support Vector Machines (SVM): SVM became popular in stock prediction because it has the ability to establish the best decision boundaries in high dimensional feature spaces. Through kernel functions, the data can implicitly be mapped by SVM to higher dimensions where linear separation can be done. This can help in the capture of non-linear relationships between the technical indicator, fundamental factor and price movement.

Random Forests and Ensemble Methods: Gradient boosting algorithms (XGBoost, LightGBM) and random forests proved to be highly effective with the combination of various decision trees. XGBoost Classifier was superior to other conventional machine learning algorithms in predictive maintenance machines as shown in the article by [2]. These ensemble techniques are ideal in dealing with mixed data types, dealing with missing values, and rank ranking the features of importance- features that can prove useful in stock prediction when the features are continuous price data to discrete market regime indicators.

The weakness of these machine learning models is that they do not have the capability to discern temporal sequence naturally. Stock prices are based on trends over days, weeks, or even months, yet conventional ML algorithms consider them separately, unless lagged features are manually added. This involves massive feature engineering and domain knowledge.

2.3.3 Deep Learning-Based Approaches

Deep learning has effectively changed the concept of stock prediction by making it possible to automatically learn hierarchical representations on raw sequential data.

Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM): RNNs brought about the idea of hidden states which store information on a time basis. Nevertheless, conventional RNNs experience vanishing as well as exploding gradient issues in the context of long sequences. Hochreiter and Schmidhuber proposed LSTM networks which overcome this limitation by using complex gating mechanisms.

A comprehensive study on predictive maintenance by [2] established that LSTM significantly outperforms Artificial Neural Networks (ANN), achieving an accuracy of 96.5% compared to ANN's 64.9%. This dramatic improvement demonstrates LSTM's superiority in handling "sequential, time-series data with dependencies that last a long time" [2].

The LSTM cell architecture consists of three gates:

$$\text{Forget Gate: } f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2)$$

$$\text{Input Gate: } i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3)$$

$$\text{Output Gate: } o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (4)$$

These gates allow the network to selectively forget other irrelevant information, update its memory with other information, and generate outputs according to filtered memory states. This ability is essential in stock prediction where price fluctuations are modulated by the historical trends in various time scales, including intraday trends and patterns of several months long.

Importantly, [2] notes that “increasing layers in ANN model could not help in improving accuracy” compared to LSTM. This serves as a cautionary insight for financial modeling: simply adding depth without the temporal memory mechanisms of LSTM is often futile. The paper also demonstrates that LSTM can effectively “identify trends and produce forecasts” when combined with anomaly detectors—a strategy directly applicable to detecting market crashes or sudden regime shifts analogous to machine “failures.”

Bidirectional LSTM (BiLSTM): BiLSTM builds on LSTM by working with sequences in either direction and enables the network to simultaneously capture both future and past contexts. This implies that in stock prediction, the model can be trained using the trends before and after a given point in time, and thus provide more accurate reversals and changes in trends.

Gated Recurrent Units (GRU): GRU offers a simpler version of LSTM that utilizes a single update gate that combines both forget and input gates. GRU is computationally simpler to use; however, it tends to have similar performance on most financial forecasting problems to LSTM.

2.3.4 Advanced Neural Network Architectures

Convolutional Neural Networks (CNN) for Time Series: Although it was initially used to process images, 1D-CNNs have been found to be useful in time-series data by identifying temporal patterns in the neighborhood. The convolutional filters employed in CNNs extract characteristics like peaks, dips as well as clusters of short-term volatility. The sequences are processed by the architecture as:

$$y_k = \sum_{j=0}^{K-1} w_j \cdot x_{t-j} + b \quad (5)$$

where the convolution window K captures neighborhood information.

Attention Mechanisms and Transformers: Transformer architectures, introduced through the “Attention Is All You Need” paper, revolutionized sequence modeling by replacing recur-

rence with self-attention mechanisms. The attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6)$$

Transformers analyze all time points simultaneously, as opposed to LSTMs that process the sequence step-by-step, and learn the historical periods that are most applicable to prediction. This makes very long sequences easier to handle, enables parallel training to run much more quickly, and the visualization of attention weights enables much better interpretability.

Graph Neural Networks (GNN): GNNs model the relationship between stocks, the effect of companies in the same industry or supply chain on each other. GNNs can spread information throughout the network structure to learn patterns across the sector and have correlation dynamics since they represent stocks as nodes, and relationships as edges.

2.3.5 Sentiment-Based Methods

Sentiment analysis has emerged as a complementary approach to purely price-based prediction, motivated by the recognition that financial markets are fundamentally information-processing systems. Sentiment methods attempt to quantify the polarity and intensity of opinions expressed in financial text—including news articles, earnings call transcripts, analyst reports, and social media posts—and use these quantified signals as predictive features.

Lexicon-Based Methods: The earliest approaches to financial sentiment analysis relied on manually constructed sentiment lexicons. VADER (Valence Aware Dictionary and sEntiment Reasoner) [3] uses a rule-based system with a human-validated sentiment dictionary to produce compound polarity scores for text. SentimentWordNet extends WordNet with positivity, negativity, and objectivity scores for each synset. These lexicon-based methods are computationally efficient and fully interpretable, but they suffer from critical limitations in financial contexts: they cannot handle context-dependent polarity shifts (e.g., “high” is positive for revenue but negative for debt), domain-specific jargon, negation scope, or complex linguistic constructions such as hedging and sarcasm.

Transformer-Based Models (BERT): The introduction of BERT (Bidirectional Encoder Representations from Transformers) [4] marked a paradigm shift in sentiment analysis. BERT’s pre-training on large general corpora via masked language modeling and next-sentence prediction produces deep contextualized representations that capture semantic nuance far beyond lexicon matching. Fine-tuned BERT models achieve state-of-the-art performance on general sentiment benchmarks. However, general-purpose BERT models trained on Wikipedia and BookCorpus perform poorly on financial text because financial language has a fundamentally different vocabulary, semantic structure, and pragmatic register compared to general English.

Domain-Adapted Models (FinBERT): FinBERT [5] addresses the domain sensitivity gap by further pre-training BERT on financial corpora including financial news, earnings call transcripts, and analyst reports. This domain adaptation significantly improves sentiment classification accuracy on financial text by exposing the model to domain-specific vocabulary and semantic patterns. FinBERT produces three-class sentiment predictions (positive, negative,

neutral) and has become the de facto standard for transformer-based financial sentiment analysis. The Financial Phrasebank dataset [6] provides a widely used benchmark of 4,840 sentences annotated by financial domain experts for training and evaluation.

Other Specialized Models: RoBERTa [7] improves upon BERT through optimized pre-training with dynamic masking, larger batches, and longer training. While not financial-domain-specific, RoBERTa’s stronger baseline performance makes it a competitive starting point for domain-adapted financial models. Ensemble approaches that combine lexicon-based scores with transformer-based predictions have also been explored, leveraging the complementary strengths of rule-based interpretability and neural contextual understanding.

Despite the progress in sentiment modeling, several critical gaps remain—including sarcasm detection, tokenization drift in financial language, temporal alignment with trading days, and uncertainty quantification—which are discussed in detail in Section 2.6.6.

2.4 Hybrid Architectures

The hybrid model came about after it was realized that financial markets cannot be viewed under one lens. Stock prices are the result of the linear behavior, non-linear behavior, market noise, market shocks, and sentiment-based behavior. The complexity of the market cannot be fully captured using either traditional statistical tools or pure deep learning ones.

2.4.1 ARIMA-LSTM Hybrid Models

The combination of ARIMA and LSTM networks is one of the first and the most powerful hybrid structures. The reasoning behind this is simple, ARIMA is the best at reflecting linear autocorrelation structures and LSTM is the best at non-linear residual patterns.

The modeling pipeline typically follows:

1. ARIMA predicts the linear component: $\hat{y}_t^{ARIMA}$
2. Compute residuals: $r_t = y_t - \hat{y}_t^{ARIMA}$
3. LSTM learns the residual component: $\hat{r}_t^{LSTM}$
4. Final prediction: $\hat{y}_t = \hat{y}_t^{ARIMA} + \hat{r}_t^{LSTM}$

The decomposition can usually result in more solid and understandable forecasts. The ARIMA component will give a baseline trend and LSTM will record deviations in a complex manner than the baseline trend.

2.4.2 CNN-LSTM Hybrid Models

Another widely studied direction combines Convolutional Neural Networks (CNNs) with LSTMs. A pivotal study by [8] explores this architecture in the context of biomedical signal processing, comparing a “Raw signal and LSTM-1D CNN” approach against a “Spectrogram and 2D CNN” method for artifact detection in electrodermal activity signals.

The study yields a profound insight directly applicable to financial data representation: **the 1D-CNN model applied to raw signals outperformed the 2D-CNN applied to visual spectrograms (Kappa 0.49 vs 0.42)** [8]. The authors attribute this to the fact that “CNNs base their knowledge on the local information,” which was better preserved in the raw 1D signal than in the global structure of the spectrogram.

This finding challenges a common trend in financial machine learning where stock price charts are converted into images for computer vision models. Instead, the research suggests that a hybrid model using **1D-CNNs to extract local volatility features from raw price series, coupled with LSTM for temporal trend analysis, represents a superior architectural choice.**

The specific configuration validated in [8]—using a kernel size of 5×5 and dropout of 0.05—provides a robust blueprint. The architecture demonstrated the best performance with:

- TPR (True Positive Rate): 0.65
- Kappa: 0.49
- AUC: 0.76

After post-processing to merge artifacts separated by under 2 seconds (analogous to smoothing in financial data), performance improved to TPR of 0.72 and Kappa of 0.50 [8].

For stock prediction, the CNN component extracts local features such as:

- Short-term price momentum patterns
- Intraday volatility clusters
- Support and resistance level formations

The LSTM component then models how these extracted features evolve through time, capturing:

- Multi-day trend persistence
- Volume-price relationships over time
- Regime transitions from bullish to bearish markets

2.4.3 Attention-Enhanced Hybrid Models

Contemporary hybrid architectures use attention mechanisms to enable the model to concentrate on the best historical periods which are significant. The most typical implementation is based on CNN or LSTM blocks to extract initial features and then Transformer blocks:

$$\text{CNN/LSTM} \rightarrow \text{Features} \rightarrow \text{Transformer} \rightarrow \text{Prediction} \quad (7)$$

Instead of the traditional approach to predicting future prices (directly learning to predict future price) the attention mechanism is trained to predict future price by considering previous time steps that are most predictive which gives better predictive accuracy and more interpretable attention weights.

2.4.4 Multimodal Hybrid Models

The most recent advancement involves integrating multiple data sources:

- **Numerical:** Historical prices, trading volumes, technical indicators
- **Textual:** Financial news, earnings reports, social media sentiment
- **Graph-based:** Inter-stock correlations, sector relationships

These models involve the encoding of each modality into separate encoding pathways and then combination of the representations to make final prediction. As an example, the BERT or GPT models work with text, the GNN with graph structures, and CNN-LSTM with time-series, and all the outputs are fused using attention-based layers.

2.4.5 Feature-Level Hybridization

Hybridization at the feature level is not necessarily paired-off with architecture. The modern forecasting systems tend to combine various types of features:

- **Technical Indicators:** MACD, RSI, Bollinger Bands, moving averages
- **Volatility Measures:** VIX, GARCH-based volatility estimates
- **Macroeconomic Signals:** Inflation rates, interest rates, GDP growth
- **Sector-Level Dependencies:** Industry performance indices, supply chain relationships

The system is taught to learn relationships, which cannot be detected solely by price, to the given richer feature set in the model. It is particularly helpful in the emerging markets where the stock movement is more affected by macroeconomic announcements compared to the developed ones.

2.4.6 Price + Sentiment Hybrid Models

A growing body of work integrates sentiment signals from financial text into price prediction models, creating hybrid systems that combine numerical and textual information. These systems recognize that price-only models, regardless of architectural sophistication, are structurally blind to exogenous information shocks until those shocks are already reflected in historical prices.

LSTM + Sentiment Models: Early sentiment-trading integration work combined LSTM networks with sentiment features derived from news headlines or social media posts. In these approaches, sentiment scores—typically produced by VADER [3] or a fine-tuned BERT model [4]—are concatenated with OHLCV features and fed into an LSTM network. While these systems demonstrate improvements over price-only LSTMs, particularly in directional accuracy, they suffer from the limitations of their constituent components: LSTM’s sequential processing bottleneck, naive sentiment scoring without sarcasm correction, and simple timestamp matching without principled temporal alignment.

FinBERT + Gradient Boosting Integration: More recent work combines domain-adapted sentiment from FinBERT [5] with gradient boosting models such as XGBoost. The advantage of this approach is that XGBoost naturally handles mixed feature types (continuous price features alongside continuous or categorical sentiment features) and provides built-in feature importance rankings that reveal the relative contribution of sentiment signals versus price-based features. However, most existing implementations use raw sentiment scores without uncertainty quantification, ignore the temporal alignment problem between news publication times and trading day boundaries, and do not address sarcasm or irony in financial text.

End-to-End Multimodal Architectures: The most recent approaches use end-to-end deep learning architectures that jointly encode text (via BERT or GPT-based models) and price series (via LSTM or Transformer encoders), fusing the representations through attention-based layers before producing a final prediction. While architecturally elegant, these models are computationally expensive, difficult to interpret, and often require large amounts of paired text-price training data that may not be available for all stocks or markets. The opacity of end-to-end approaches also makes it difficult to diagnose whether sentiment signals are genuinely contributing to prediction quality.

Common Deficiencies in Sentiment-Trading Hybrids: Despite the growing interest, existing sentiment-trading hybrid models share several deficiencies: (1) sarcasm detection is rarely addressed, despite its prevalence in financial commentary; (2) temporal alignment between news publication timestamps and trading day boundaries is handled naively, often introducing data leakage; (3) sentiment features are typically raw point estimates without confidence weighting or uncertainty quantification; (4) the frequency mismatch between irregularly arriving news and regularly spaced price data is not formally resolved; and (5) evaluation focuses on price accuracy metrics rather than risk-adjusted trading performance metrics such as the Sharpe ratio.

2.5 Comparative Analysis of Hybrid Models

Understanding the trade-offs between different hybrid architectures is crucial for selecting the appropriate model for specific forecasting tasks. Table 2 provides a comprehensive comparison of major hybrid approaches.

2.5.1 Why Hybrid Models Represent the Future

Hybrid models are becoming increasingly important for several fundamental reasons:

- 1. Financial Data is Multi-Scale:** Short-term noise, medium-term trends, and long-term cycles all exist simultaneously in financial markets. A single model architecture rarely captures all three timescales effectively. Hybrid models can assign different components to handle different temporal resolutions.

- 2. Markets are Influenced by Many Types of Signals:** Price, volume, news, sentiment, global indices, and sector relationships all play roles in price formation. Hybrid models are naturally suited to merge these diverse information sources through multi-modal learning architectures.

Table 2: Comprehensive Comparison of Hybrid Models

Hybrid Model		Components Used	Key Strengths	Limitations	Best Case	Use-Case
ARIMA + LSTM	+	ARIMA (linear), LSTM (non-linear)	Separates linear and non-linear behavior; good stability; interpretable	Struggles during sudden market shifts; limited long-range memory	Medium-term forecasting; markets with clear trend + seasonal patterns	
CNN + LSTM		CNN (local pattern extraction), LSTM (sequential modeling)	Captures short-term patterns + long-term dependencies; strong on high-frequency data	Sensitive to noise; heavy tuning required	High-frequency trading, intra-day patterns	
CNN + GRU		CNN (feature extraction), GRU (simplified RNN)	Faster than CNN-LSTM, fewer parameters, easier to train	Slightly less expressive than LSTM	Fast inference or low-resource environments	
Bi-LSTM + Attention	+	Bi-LSTM (bidirectional memory), Attention (focus on important time points)	Captures forward + backward context; highlights important events	Higher training cost; still sequential	Medium-sequence forecasting with strong context	
LSTM + Transformer	+	LSTM (local sequence), Transformer (long-range dependencies)	Balances sequential learning with global attention; strong generalization	Computationally heavy; requires large datasets	Long-horizon forecasting; markets with irregular patterns	
CNN + LSTM + Transformer		CNN (local), LSTM (temporal), Transformer (global attention)	Strongest hybrid; captures all three market scales (micro, medium, macro)	High complexity; may overfit without strong regularization	Most modern, most accurate general-purpose hybrid	
Price + Sentiment Model		LSTM/Transformer for numerical data, Text encoder for sentiment	Uses both technical signals and market sentiment; responds to news	Hard to align sentiment timing with price reaction	Event-driven forecasting, earnings announcements	
Feature-Level Hybrids		Technical indicators + macro data + volatility metrics	Models broader market environment; reduces reliance on price alone	Requires careful feature engineering; prone to redundancy	Multi-factor forecasting; cross-market analysis	

3. Market Behavior is Partly Linear and Partly Chaotic: Linear components (trends, seasonality) are efficiently handled by statistical models like ARIMA, whereas chaotic and non-linear dependencies (sudden jumps, regime changes) require deep learning architectures. Hybrid models leverage both paradigms.

4. Hybrid Models Reduce Risk of Model Bias: When one component fails under specific market conditions, other components can compensate. This ensemble-like behavior builds robustness during volatile events or structural breaks.

5. Theoretical Justification—Error Decomposition: If the true data-generating process follows:

$$y_t = L_t + N_t + \epsilon_t \quad (8)$$

where L_t represents linear structure, N_t represents non-linear behavior, and ϵ_t represents noise, no single model can perfectly learn both L_t and N_t simultaneously. Hybrid architectures allow:

- ARIMA $\rightarrow$ learns L_t
- LSTM/CNN $\rightarrow$ learns N_t
- Transformer $\rightarrow$ learns long-range dependencies across both

This decomposition creates a more complete representation of the underlying data generation process. In most recent studies (2021–2024), hybrid CNN–LSTM–Transformer models have consistently outperformed single-model approaches in accuracy, stability, and robustness across diverse market conditions.

2.6 Limitations of Existing Systems

2.6.1 Statistical Model Constraints

Linearity Assumptions: As noted in the ARIMA analysis [1], statistical models require strict stationarity achieved through differencing. This often results in the loss of valuable long-term trend information. In highly volatile stock markets characterized by regime changes and structural breaks, the linear framework becomes fundamentally inadequate.

Feature Independence: Traditional models like ARIMA assume independence of explanatory variables when extended to multivariate forms (VARIMA, VAR). In reality, stock market features exhibit complex interactions—volume changes affect momentum, which influences volatility, creating feedback loops that linear models cannot capture.

2.6.2 Machine Learning Limitations

Temporal Ignorance: As highlighted earlier, classical ML algorithms (SVM, Random Forest, XGBoost) treat each time point independently unless manually provided with engineered lagged features. This requires extensive domain expertise and may miss complex temporal dependencies.

Inadequacy of Single Deep Learning Models: As evidenced by [2], while LSTM outperforms ANN (96.5% vs 64.9% accuracy), reliance on a single architecture limits the model's

ability to capture both local feature variations and global trends simultaneously. Pure LSTM models may struggle with:

- Very long sequences leading to gradient degradation
- Inability to capture multi-scale temporal patterns efficiently
- Lack of explicit mechanisms for feature extraction at different resolutions

2.6.3 Feature Representation Challenges

Data Representation Fallacy: A major gap exists in how financial data is encoded for neural networks. The finding by [8] that 1D-CNNs on raw signals outperform 2D-CNNs on spectrograms (Kappa 0.49 vs 0.42) highlights a flaw in approaches that convert stock charts into images. Many systems attempt to use computer vision techniques on candlestick chart images, but this introduces:

- Loss of numerical precision through image quantization
- Artificial spatial correlations not present in the original data
- Increased computational complexity without performance gains

There is a clear lack of models that properly leverage 1D-CNNs for local volatility extraction directly from raw price sequences.

2.6.4 Computational and Real-Time Constraints

Deep learning models, particularly complex hybrid architectures, demand significant computational resources. The study by [8] used early stopping with a threshold of 30 epochs and careful hyperparameter tuning, yet still required substantial training time. For high-frequency trading or real-time prediction systems, latency becomes a critical constraint that many sophisticated models cannot satisfy.

2.6.5 Overfitting and Generalization Issues

Financial markets are non-stationary—patterns that work in one market regime may fail in another. Models trained on historical bull markets often perform poorly during bear markets or crisis periods. The COVID-19 pandemic illustrated this dramatically, as many prediction systems trained on pre-2020 data failed catastrophically when market dynamics shifted.

2.6.6 Sentiment Analysis and Integration Gaps

Beyond the limitations of purely price-based systems, the application of sentiment analysis to financial prediction introduces its own set of well-documented gaps:

Sarcasm and Irony Detection: Financial commentary—particularly in analyst reports, financial blogs, and social media—frequently employs sarcasm and irony. Standard sentiment

models classify sarcastic positive statements as genuinely positive, inverting the true polarity. For example, the headline “Oh great, another revenue miss” would be classified as positive by a naive model, directly misleading the downstream prediction system. Dual-head architectures combining a primary sentiment head with a secondary sarcasm detection head have been proposed in the NLP literature but are not yet standard practice in financial sentiment pipelines [5].

Tokenization Drift: Pre-trained transformer models use tokenizers trained on general text corpora. When applied to financial text, these tokenizers fragment domain-specific tokens—ticker symbols (\$AAPL), financial ratios (P/E, EV/EBITDA), regulatory language (Form 10-K, material adverse change)—into meaningless subword units. This *tokenization drift* degrades the model’s ability to represent financial entities accurately and requires custom vocabulary extensions or domain-specific tokenizer fine-tuning to resolve.

Domain Sensitivity: General-purpose language models trained on Wikipedia and Book-Corpus assign different semantic distributions to words than financial contexts require. For instance, “volatility” carries negative sentiment in general text but is a neutral technical descriptor in finance, while “aggressive” describes a negative behavior in general text but a positive growth strategy in business contexts. Domain-adapted models such as FinBERT [5] mitigate this gap but do not fully eliminate it, particularly for rapidly evolving financial terminology and novel financial instruments.

Temporal Alignment: News articles are published at irregular timestamps—often after market hours, on weekends, or during trading sessions. Naively aligning news timestamps with the same calendar date introduces data leakage: a news article published at 11 PM EST on Friday should influence Monday’s trading, not Friday’s. Most existing systems use naive timestamp matching without accounting for market calendar effects, leading to artificially inflated training performance that does not generalize to real-time deployment.

Uncertainty Quantification: Standard neural sentiment models output deterministic point estimates with no associated confidence measure. In a trading context, a sentiment score of +0.7 with 95% model confidence is fundamentally different from a score of +0.7 with 55% confidence. Ignoring prediction uncertainty leads to overconfident trading signals and inappropriate position sizing in downstream trading strategies.

Aspect-Based Sentiment Analysis (ABSA): A single news article may simultaneously carry positive sentiment about revenue growth and negative sentiment about regulatory risk. Standard sentiment models produce a single document-level score that collapses these distinct aspect-level signals into a single number, losing actionable information. Aspect-Based Sentiment Analysis with dependency parsing can extract aspect-sentiment pairs, but its integration into trading systems remains underdeveloped.

2.7 Comparative Study / Gap Analysis

2.7.1 Method Comparison: Statistical vs. Machine Learning vs. Deep Learning

Table 3: Comparative Analysis of Stock Prediction Approaches

Method	Linearity	Temporal	Complexity	Interpretability
ARIMA	Linear	Yes	Low	High
SVM/RF	Non-linear	Limited	Medium	Medium
LSTM	Non-linear	Yes	High	Low
CNN-LSTM	Non-linear	Yes	Very High	Low
Transformer	Non-linear	Yes	Very High	Medium

2.7.2 Performance Comparison from Literature

Based on the analyzed studies:

- **LSTM vs. ANN** [2]: LSTM achieved 96.5% accuracy compared to ANN's 64.9%, demonstrating the critical importance of temporal modeling mechanisms.
- **1D-CNN-LSTM vs. 2D-CNN** [8]: Raw signal processing with 1D-CNN-LSTM achieved Kappa 0.49 vs. 0.42 for spectrogram-based 2D-CNN, highlighting the importance of proper data representation.
- **ARIMA applicability** [1]: Effective for linear trend and seasonality detection but requires stationarity and fails under regime changes.

2.7.3 Integration Gaps and Research Deficiencies

Although the modern stock prediction methods have come a long way, the contemporary studies have a number of critical flaws that restrict the feasibility and real-world performance:

1. Difficulty Incorporating Multiple Data Types Efficiently

Even though numerous hybrid models purport to combine sentiment analysis, the existing approaches continue to encounter basic multimodal issues:

- **Pipeline Integration:** Combining news, social media, macroeconomic data, and price data into a single coherent pipeline remains problematic. Most approaches process these modalities separately and perform late fusion, losing valuable inter-modal interactions.
- **Frequency Mismatch:** Handling different data frequencies poses significant technical challenges. News and social media arrive irregularly and asynchronously, while price data is continuous and regular. Current architectures lack robust mechanisms for temporal alignment across these disparate frequencies.
- **Semantic-Temporal Alignment:** Aligning textual signals (news sentiment) with time-series signals (price movements) is non-trivial. The time lag between news publication and market reaction is variable and context-dependent, and most models use naive timestamp matching rather than learned alignment strategies.

2. Absence of Adaptive Learning

Financial markets are non-stationary by nature - a model that has been trained on the data of the previous month may go haywire on the data of the current month because of regime changes, changes in policies, or even events in the world. Limitations The main limitations are:

- **Static Training Paradigm:** Most hybrid models follow a train-once-deploy-forever approach. They cannot update themselves incrementally without full retraining, making them vulnerable to concept drift.
- **Regime Change Blindness:** Markets transition between bullish, bearish, and volatile regimes. Current models lack explicit mechanisms to detect regime shifts and adapt their behavior accordingly.
- **Computational Overhead:** Even when researchers acknowledge the need for adaptation, the computational cost of retraining complex hybrid models (especially those involving Transformers or multi-modal fusion) makes continuous learning impractical for production systems.

3. Lack of Risk-Aware Prediction

The majority of hybrid models are concerned only with the direction of the prices or their values, but they do not take into account the uncertainty and risk of their predictions:

- **Point Predictions Only:** Standard neural networks output deterministic point estimates without confidence intervals or probabilistic distributions. This makes it impossible for traders to assess prediction reliability.
- **No Risk Quantification:** Critical risk metrics—drawdown potential, Value-at-Risk (VaR), conditional volatility, tail risk—are absent from model outputs. Deep learning architectures are not inherently designed to quantify uncertainty.
- **Architectural Complexity:** Hybrid models are already complex. Adding risk prediction requires separate architectural components (e.g., Bayesian layers, dropout-based uncertainty estimation, or ensemble variance), further increasing training difficulty and computational requirements.
- **Volatility-Return Decoupling:** Financial risk fundamentally depends on future volatility, which is highly unpredictable and influenced by different factors than price movements. Very few papers attempt to jointly model volatility forecasting and price forecasting within a single hybrid architecture.
- **Evaluation Metric Bias:** Most research optimizes for accuracy-based metrics (MAE, RMSE, directional accuracy) rather than risk-adjusted metrics (Sharpe ratio, information ratio, maximum drawdown). This creates a mismatch between research objectives and practical trading requirements.

4. Sentiment-Trading Integration Deficiencies

Beyond the gaps internal to sentiment models (Section 2.6.6), there are additional deficiencies specific to the integration of sentiment analysis into trading pipelines:

- **Frequency Mismatch:** Price data arrives at regular daily (or intraday) intervals, while news arrives asynchronously and at irregular frequencies. Robust temporal aggregation strategies—such as confidence-weighted daily averaging with formal trading-day assignment—are required but rarely implemented in existing work.
- **Feature Redundancy:** Naively concatenating raw sentiment scores with technical indicators risks introducing correlated features that degrade model performance through multicollinearity. Systematic feature selection with Variance Inflation Factor ($VIF > 10$ removal) is rarely applied in sentiment-trading systems.
- **Regime Sensitivity:** The relationship between sentiment and price movement is not stationary. In a strongly trending bull market, negative sentiment may have less impact than in a volatile or bearish regime. Current models assume a fixed sentiment-price relationship, ignoring regime-dependent dynamics that require adaptive weighting.
- **Risk-Aware Evaluation:** Most sentiment-trading systems optimize for accuracy-based metrics (MAE, RMSE, directional accuracy) rather than risk-adjusted trading metrics (Sharpe ratio, maximum drawdown, Value-at-Risk). This creates a fundamental gap between research objectives and practical trading requirements.

Summary of Critical Gaps:

- Statistical models (ARIMA, GARCH) are well-understood but limited in capability
- Deep learning models (LSTM, CNN) are powerful but often used in isolation
- Hybrid approaches exist but lack standardization in architecture design
- True multimodal integration across heterogeneous data sources remains under-developed
- Adaptive learning mechanisms for handling concept drift are largely absent
- Risk-aware prediction with uncertainty quantification is critically missing
- Sentiment models suffer from sarcasm blindness, tokenization drift, domain sensitivity, and lack of confidence estimation
- Sentiment-trading integration is hampered by temporal misalignment, frequency mismatch, feature redundancy, and regime-insensitive weighting

2.8 Summary of Literature Review

2.8.1 Key Findings

1. **Evolution of Methodology:** Stock prediction has progressed from simple statistical models (ARIMA, GARCH) through machine learning (SVM, RF) to sophisticated deep learning (LSTM, CNN, Transformers) and hybrid architectures.

Table 4: Formal Summary of Identified Research Gaps in Stock Prediction and Sentiment Analysis

Gap Category	Specific Gap	Current Status	Impact on Prediction
Sentiment Models	Sarcasm and irony detection	Dual-head proposed, not yet standard	Polarity inversion errors
Sentiment Models	Tokenization drift in financial text	Domain vocabulary extension needed	Degraded entity representation
Sentiment Models	No uncertainty quantification	Deterministic outputs only	Overconfident trading signals
Sentiment Models	Document-level scoring only (no ABSA)	Aspect extraction underdeveloped	Lost aspect-level signals
Integration	Temporal alignment	Naive timestamp matching	Data leakage in training
Integration	Frequency mismatch	No formal aggregation strategy	Inconsistent feature vectors
Integration	Feature redundancy	No systematic VIF control	Multicollinearity and overfitting
Modeling	Static training paradigm	No adaptive / online learning	Degraded post-shift performance
Modeling	Regime-blind sentiment weighting	Fixed sentiment-price relationship	Misweighted signals across regimes
Evaluation	Accuracy-focused metrics only	RMSE/MAE optimization	Gap between research and practice

2. **Temporal Modeling is Critical:** As demonstrated by [2], models with explicit temporal mechanisms (LSTM) dramatically outperform those without (ANN), highlighting that sequence modeling is non-negotiable for financial forecasting.
3. **Proper Data Representation Matters:** The insight from [8] that 1D-CNN on raw signals outperforms 2D-CNN on transformed representations challenges common practices and suggests that raw time-series processing should be preferred over image-based approaches.
4. **Hybrid Models Show Promise:** Combining complementary strengths—ARIMA’s linearity with LSTM’s non-linearity, CNN’s feature extraction with LSTM’s sequence modeling—consistently yields better performance than single-method approaches.
5. **Significant Gaps Remain:** Issues of non-stationarity, regime changes, proper feature representation, computational efficiency, and multimodal integration remain largely unsolved.
6. **Sentiment Analysis Extends the Frontier:** Sentiment analysis methods have evolved from lexicon-based approaches (VADER) to domain-adapted transformers (FinBERT [5]), enabling quantification of the informational environment that drives market behavior. When integrated into hybrid models, sentiment features provide complementary signals that price-only models structurally cannot capture—particularly around earnings surprises, macroeconomic announcements, and shifts in public opinion.
7. **Critical Gaps in Sentiment Integration Persist:** Despite the promise of sentiment-aware models, the literature reveals six major gaps in sentiment analysis (sarcasm detection, tokenization drift, domain sensitivity, temporal alignment, uncertainty quantification, and aspect-based analysis) and four integration-level deficiencies (frequency mismatch, feature redundancy, regime sensitivity, and risk-aware evaluation). These gaps represent the most significant obstacles to deploying robust sentiment-augmented trading systems, as summarized in Table 4.

2.8.2 Conclusion

The literature demonstrates definitive advancement in hybrid, multimodal, and attention-based stock prediction architectures, while simultaneously revealing that sentiment-aware models represent a critical and under-explored frontier. Nevertheless, there are still substantial gaps in research, especially in:

- Developing standardized hybrid architectures that optimally combine statistical rigor with deep learning flexibility
- Creating adaptive systems that recognize and adjust to market regime changes
- Properly leveraging 1D-CNN for local feature extraction as suggested by [8]

- Integrating diverse data modalities (numerical, textual, graph-based) in a principled manner
- Balancing model complexity with interpretability for practical deployment
- Addressing sarcasm, tokenization drift, and domain sensitivity in financial sentiment pipelines to produce reliable sentiment signals
- Designing principled temporal alignment mechanisms that map news publication timestamps to actionable trading days without information leakage
- Incorporating uncertainty quantification into sentiment features so that downstream models can distinguish confident signals from noisy ones
- Moving evaluation beyond accuracy-based metrics toward risk-adjusted trading metrics that reflect practical deployment requirements

These identified gaps—spanning both price-based and sentiment-based methodologies—motivate the development of a sentiment-aware hybrid model that addresses the most critical deficiencies documented in this review, particularly around sarcasm-corrected sentiment scoring, principled temporal alignment, and structured sentiment feature engineering.

3 Problem Identification

3.1 Problem Statement

Although decades of research have already been made and sophisticated machine learning techniques have been developed, the successful prediction of the price of stocks is an uphill task. The main problem is due to the specifics of financial markets, which display:

- **Non-stationarity:** Statistical properties change over time due to evolving market conditions, regulatory changes, and macroeconomic shifts.
- **High dimensionality:** Prices are influenced by thousands of factors including company fundamentals, sector trends, macroeconomic indicators, and investor sentiment.
- **Non-linearity:** Relationships between variables are complex and often change based on market regimes.
- **High noise-to-signal ratio:** Random fluctuations and speculative trading introduce substantial noise.
- **Extreme events:** Financial crises, pandemics, and geopolitical events create outliers that violate assumptions of most models.
- **Exogenous information blindness:** Price-only models have no mechanism to incorporate the impact of news events, earnings announcements, or shifts in public sentiment until these signals have already been absorbed into historical price data, introducing an inherent reaction lag.

Current strategies cannot cover these problems at the same time. The conventional statistical approaches are capable of dealing with linearity and interpretability, but fail to deal with complicated non-linear processes. Deep learning models do not interpret but are able to capture non-linearity and have weaknesses in terms of regime change. Existing hybrid models are yet to be promising with inefficient architectural designs and inappropriate data representation strategies. Furthermore, the vast majority of existing approaches—whether statistical, machine learning, or hybrid—operate exclusively on historical price and volume data, rendering them structurally blind to exogenous information shocks such as surprise earnings reports, regulatory actions, or macroeconomic announcements. A model that cannot process textual signals from financial news is fundamentally limited in its ability to anticipate, rather than merely react to, market-moving events.

3.2 Context and Background of the Problem

The stock market is a financial feed in the economy and it is also one of the biggest ways of wealth creation and the distribution of the funds in the modern elective economies. These consequences of correct price forecasting are extensive:

- **For Investors:** Better predictions enable informed decision-making, portfolio optimization, and risk management.
- **For Institutions:** Financial institutions use predictions for algorithmic trading, hedging strategies, and market-making activities.
- **For Regulators:** Understanding price dynamics helps in detecting market manipulation, maintaining stability, and formulating policies.
- **For Researchers:** Stock prediction serves as a proving ground for time-series forecasting methods applicable to diverse domains.

With the emergence of the big data and the power of computers it has opened an opportunity to create more complex systems of prediction. Nevertheless, the intrinsic issues which include non-stationarity, complexity and unpredictability remain. This is what COVID-19 demonstrated: the majority of prediction models that had been trained on the data before the pandemic collapsed when the market situation radically changed in March 2020.

3.3 Need and Importance of Solving the Problem

3.3.1 Economic Impact

Stock markets around the world are tens of trillions of dollars in their market capitals. Even the slight changes in prediction level can be converted to huge financial benefits or loss prevention. To institutional investors whose assets are in the billions, a 1 percent improvement in prediction means millions of dollars in better returns or less risk.

3.3.2 Scientific Advancement

Machine learning methods are challenged by prediction of stocks. Innovations in this field are frequently reused in other time-series problems in healthcare (patient monitoring), climate science (weather forecasting), energy (demand prediction) and industrial systems (predictive maintenance).

3.3.3 Market Efficiency

More precise pre predictions lead to efficiency in the market because they add up information faster in the prices. This lessens arbitrage possibilities, narrows bid-ask spreads and enhances price discovery -all to the benefit of the wider economy.

3.3.4 Risk Management

The effects of financial crisis on the society are devastating. The 2008 financial crisis in itself caused millions of people to lose their jobs and billions in wealth destruction. Improved prediction and early warning mechanisms can aid in the detection of bubbles, identification of systemic risks and preemptive interventions.

3.4 Scope and Project Objectives

3.4.1 Scope

This study aims at coming up with a better hybrid structure of predicting stock prices to overcome the limitations presented. The scope includes:

- Designing a hybrid model integrating ARIMA for linear trend extraction, 1D-CNN for local feature detection, and LSTM for temporal sequence modeling.
- Validating the model on major stock indices and individual stocks across different market conditions.
- Comparing performance against baseline models (pure ARIMA, pure LSTM, existing hybrids).
- Analyzing interpretability through attention mechanisms and feature importance.
- Designing a domain-adapted sentiment analysis pipeline that processes financial news through sarcasm-corrected, confidence-weighted scoring with principled temporal alignment to trading day boundaries.
- Integrating structured sentiment features into the hybrid SARIMA-XGBoost model and evaluating the augmented system against the price-only baseline across error-based and trading-relevant metrics.

3.4.2 Objectives

The specific objectives are:

1. **Develop a principled hybrid architecture** that optimally combines the strengths of ARIMA (linear modeling), 1D-CNN (local feature extraction), and LSTM (long-term dependency capture).
2. **Validate the architectural insight from [8]** that 1D-CNN on raw signals outperforms image-based approaches by applying it to financial time-series.
3. **Address the single-model limitation identified in [2]** by creating a multi-component system that captures both local variations and global trends.
4. **Incorporate proper stationarity handling** as highlighted by ARIMA studies [1] while avoiding information loss through adaptive preprocessing.
5. **Evaluate robustness** across different market conditions (bull markets, bear markets, high volatility periods) to ensure generalization.
6. **Balance complexity with interpretability** through attention mechanisms and ablation studies that reveal component contributions.

7. **Design a domain-adapted sentiment analysis pipeline** that addresses documented gaps in financial sentiment analysis, including sarcasm detection via a dual-head classification architecture, tokenization drift through vocabulary extension, and temporal alignment of asynchronous news timestamps to actionable trading days.
8. **Engineer structured sentiment features**—including confidence-weighted daily sentiment, sentiment momentum, rolling sentiment volatility, news volume, and sentiment-price divergence—and integrate them into the hybrid model’s feature space while controlling for multicollinearity through variance inflation factor (VIF) analysis.
9. **Evaluate the sentiment-augmented hybrid model** against the price-only baseline using both conventional error metrics (RMSE, MAE) and trading-relevant metrics (directional accuracy, Sharpe ratio) to quantify the marginal value of sentiment information.

3.5 Anticipated Challenges

3.5.1 Data Quality and Availability

The quality of financial data is quite different. Whereas larger indices have clean, high-frequency data, smaller stocks can have gaps, errors, or only a short history. Incorrectly preprocessing the missing data, outliers, and differences in the temporal resolution of various features may be critical.

3.5.2 Non-stationarity and Regime Changes

Financial markets are non-stationary as they are established in the literature review. A model, which is trained using the data between the years 2010-2019, might not be a good predictor in 2020 because of the regime beat by the pandemic. This is a challenge to come up with adaptive mechanisms or ensemble approaches that can cope with regime changes.

3.5.3 Overfitting vs. Generalization

Due to their millions of parameters, deep learning models can readily overfit training data, at least when the training window has certain patterns (e.g. a long bull market). To get the model to generalize to the unobservable markets, careful regularization, validation schemes, and perhaps, the addition of domain knowledge constraints are necessary.

3.5.4 Computational Resources

It is computationally costly to train complex hybrid models using ARIMA feature preprocessing, CNN feature extraction and LSTM sequence modelling on thousands of stocks. There is an engineering problem between trying to strike a balance between model sophistication and the practical deployment (inference time, memory footprint) constraints.

3.5.5 Evaluation Metrics

Measures such as MAE or RMSE might not encompass utility in trading. A model that has greater RMSE and greatly improved the directional accuracy (at least predicting an up/down move correctly) can prove more useful as a trading strategy. It is necessary to determine proper evaluation systems to indicate the utility in the real world.

3.5.6 Interpretability and Trust

Financiers tend to hesitate to use black-box models in making high-stakes decisions. The balancing act of ensuring an interpretable performance without compromising the performance is a sensitive matter by incorporating attention mechanisms, feature importance analysis and ablation studies.

3.5.7 Benchmark Comparison

Comparative fairly against the current methods needs proper designing of the experiment. Various papers work with various datasets, time frames, and assessment procedures that complicate their face to face comparison. It is significant to set reproducible benchmarks in order to validate contributions.

3.5.8 Sentiment Data Challenges

Integrating textual sentiment data into a quantitative prediction pipeline introduces several domain-specific challenges beyond those encountered in standard numerical time-series forecasting:

- **Temporal alignment:** Financial news articles are published asynchronously—during trading hours, after market close, on weekends, and on holidays. Naively mapping publication timestamps to the same calendar date risks data leakage, where future-looking information contaminates the training signal. A principled alignment function must map each article to the next actionable trading day.
- **Sarcasm and irony detection:** Financial commentary, particularly on social media and in analyst blogs, frequently employs sarcasm (e.g., “Oh great, another revenue miss”). Standard sentiment models classify such statements at face value, inverting the true polarity. Robust detection of sarcastic intent is necessary to avoid systematic sentiment misclassification.
- **Domain-specific language:** Financial text contains specialised terminology (e.g., ticker symbols, financial ratios such as P/E and EV/EBITDA, regulatory references) that general-purpose tokenizers fragment into meaningless subword units. This tokenization drift degrades the quality of sentiment representations and requires vocabulary extension or domain-specific fine-tuning.

- **Irregular news frequency:** Unlike price data, which arrives at regular daily intervals, news articles arrive sporadically. Some trading days may have dozens of relevant articles while others have none, necessitating robust aggregation strategies and explicit handling of missing-sentiment days (e.g., binary indicators for no-news periods).
- **Uncertainty quantification:** Most sentiment models produce deterministic point estimates with no confidence measure. In a trading context, distinguishing between high-confidence and low-confidence sentiment signals is essential to avoid overweighting unreliable scores in the downstream prediction model.
- **Regime-dependent sentiment–price coupling:** The relationship between news sentiment and subsequent price movement is not stationary. In a strong bull market, negative sentiment may have a muted impact on prices, whereas in a volatile or bearish regime the same signal may drive significant price dislocations. Models must account for this non-stationarity in the sentiment–price relationship.

4 Problem Methodology

The following chapter lays out the specific way that we will predict the price of hybrid stocks. We report the entire pipeline of data collection to feature engineering and development to model and assessment, and give a detailed blueprint of our implementation.

4.1 System Architecture Overview

The stock prediction system is based on the alayered architecture and incorporates the traditional statistical techniques, machine learning algorithms, and deep learning models to identify both the linear and non-linear tendencies of stock prices. Figure 1 in the appendix depicts the entire system architecture of data flow starting with raw stock data up to preprocessing, feature engineering, model training, and prediction output.

The architecture consists of seven primary components:

1. **Data Source:** Historical S&P 500 stock data from Yahoo Finance API and financial news articles from multiple sources
2. **Data Collection & Preprocessing:** Data cleaning, missing value handling, and train-test splitting
3. **Feature Engineering:** Creation of technical indicators, temporal features, and lag variables
4. **Sentiment Analysis Pipeline:** Domain-adapted FinBERT sentiment scoring with dual-head sarcasm correction, temporal alignment, and confidence-weighted aggregation
5. **Model Development:** Implementation of multiple prediction models (SARIMA, Prophet, XGBoost, BiLSTM+Attention)
6. **Hybrid Ensemble:** Combination of SARIMA for trend capture and XGBoost for sentiment-augmented residual learning
7. **Prediction Output:** Price forecasts with model comparison and evaluation metrics

Such a multi-model strategy enables us to use the advantages of each methodology and counterbalance the weakness of the other. The integration of a dedicated sentiment analysis pipeline provides the system with information about exogenous market-moving events that price-only features cannot capture.

4.2 Data Collection and Preprocessing

4.2.1 Data Source and Characteristics

We utilize the `yfinance` Python library to retrieve historical stock data for S&P 500 constituent companies. For each stock, we collect:

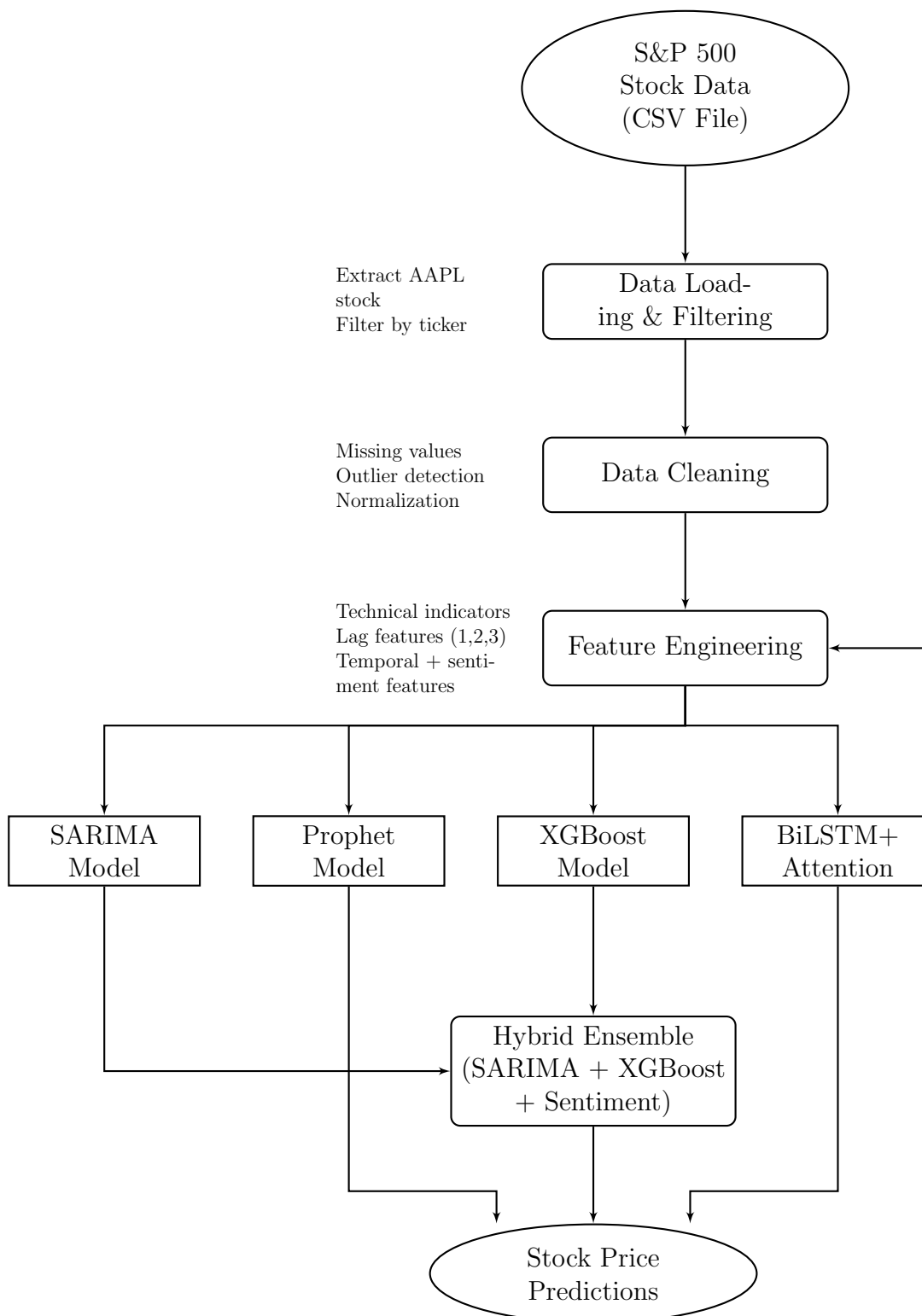


Figure 1: Complete System Architecture for Sentiment-Augmented Hybrid Stock Prediction

- **OHLCV Data:** Open, High, Low, Close prices, and trading Volume
- **Temporal Range:** Multi-year historical data to capture various market conditions
- **Frequency:** Daily stock prices with timestamps

The raw data has the following major attributes:

- **Date:** Trading date (index)
- **Open:** Opening price for the trading day
- **High:** Highest price during the trading day
- **Low:** Lowest price during the trading day
- **Close:** Closing price (primary target variable)
- **Volume:** Number of shares traded
- **Adj Close:** Adjusted closing price accounting for corporate actions

4.2.2 Data Cleaning

There are some vital steps that our data preprocessing pipeline realizes:

Missing Value Handling: There is a high chance of missing data in financial time-series data because of market holidays, suspension of trading, or data collection mistakes. We employ:

- **Forward Fill:** For single-day gaps, propagate the last available value
- **Interpolation:** For multi-day gaps, apply linear interpolation to maintain smooth transitions
- **Removal:** Discard stocks with excessive missing data (>5% of total observations)

Outlier Detection and Treatment: Radical price changes may cause model training to be inaccurate. We implement:

$$\text{Outlier} = |x_t - \mu| > 3\sigma \quad (9)$$

where μ represents the rolling mean and σ represents the rolling standard deviation over a 20-day window.

Normalization: In order to achieve a numerically stable model convergence we use Min-Max scaling:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (10)$$

4.2.3 News Data Collection

Financial news articles are collected from publicly available sources to serve as input for the sentiment analysis pipeline. Sources include Yahoo Finance News, Reuters financial RSS feeds, and the Financial Modeling Prep news API. For each article, we collect the following data fields:

- **Headline:** Title of the article
- **Body:** Full article text where available; headline-only for paywalled sources
- **Publication Timestamp:** Exact datetime of publication in UTC
- **Source:** Publisher identifier (e.g., Reuters, Bloomberg, Yahoo Finance)
- **Ticker Mention:** Binary indicator of whether the article explicitly mentions the target stock ticker (e.g., AAPL)

The news corpus spans the same temporal range as the price data (2010–2025). Article volume varies considerably over time: high-profile earnings announcements or market-moving events may generate dozens of articles per day, while quiet trading periods may produce none. This variation is explicitly captured through normalized news volume features in the sentiment pipeline. Articles are stored with their original UTC timestamps to enable the principled temporal alignment procedure described in Section 4.3.5.

4.2.4 Train-Test Split

We use a temporal split strategy in order to preserve the sequence of time-series information:

- **Training Set:** First 80% of the chronological data
- **Testing Set:** Final 20% for out-of-sample evaluation
- **Validation:** Time-series cross-validation with expanding window

This will eliminate data leakage and allow realistic evaluation which is a realistic representation of actual trading situations where the future data is unknown.

4.3 Feature Engineering

The so-called feature engineering is where the domain knowledge is coded to the model. The features we implement generate three classes of features that profiled various facets of marketing behavior.

4.3.1 Technical Indicators

Technical analysis indicators are used to measure momentum, trend, volatility and volume. Some of the indicators that we apply are as follows with the help of the Python library of technical analysis:

1. Moving Average Convergence Divergence (MACD):

$$\text{MACD} = \text{EMA}_{12}(\text{Close}) - \text{EMA}_{26}(\text{Close}) \quad (11)$$

$$\text{Signal Line} = \text{EMA}_9(\text{MACD}) \quad (12)$$

$$\text{MACD Histogram} = \text{MACD} - \text{Signal Line} \quad (13)$$

MACD shows the direction of the trend and momentum through comparisons of the short-term average and the long-term average which are exponential moving averages.

2. Relative Strength Index (RSI):

$$\text{RSI} = 100 - \frac{100}{1 + \frac{\text{Avg Gain}_{14}}{\text{Avg Loss}_{14}}} \quad (14)$$

RSI is a momentum scale ranging between 0 and 100, where scores of above 70 signal overbought markets and those of below 30 signal oversold markets.

3. Bollinger Bands:

$$\text{Middle Band} = \text{SMA}_{20}(\text{Close}) \quad (15)$$

$$\text{Upper Band} = \text{Middle Band} + 2 \times \sigma_{20} \quad (16)$$

$$\text{Lower Band} = \text{Middle Band} - 2 \times \sigma_{20} \quad (17)$$

Bollinger Bands are used to gauge the price volatility based on the 20 days moving average.

4. Simple Moving Averages (SMA):

$$\text{SMA}_n = \frac{1}{n} \sum_{i=0}^{n-1} \text{Close}_{t-i} \quad (18)$$

We calculate SMAs for multiple windows ($n = 5, 10, 20, 50, 200$) to capture different trend timescales.

5. Exponential Moving Average (EMA):

$$\text{EMA}_t = \alpha \cdot \text{Close}_t + (1 - \alpha) \cdot \text{EMA}_{t-1} \quad (19)$$

$$\text{where } \alpha = \frac{2}{n + 1} \quad (20)$$

EMA also places more emphasis on the current prices; hence it is more sensitive to new information than SMA.

Additional Technical Indicators:

- **Stochastic Oscillator:** Measures momentum by comparing closing price to price range

- **Average True Range (ATR):** Quantifies volatility
- **On-Balance Volume (OBV):** Cumulative volume indicator for price trend confirmation
- **Commodity Channel Index (CCI):** Identifies cyclical trends

4.3.2 Temporal Features

There are high calendar effects in market behavior. We derive time characteristics to reflect such patterns:

- **Day of Week:** One-hot encoded (Monday=0, ..., Friday=4)
- **Month:** One-hot encoded (January=1, ..., December=12)
- **Quarter:** Business quarters (Q1, Q2, Q3, Q4)
- **Is Month End:** Binary indicator for month-end trading effects
- **Is Quarter End:** Binary indicator for quarter-end rebalancing

These features enable models to learn the “January effect,” “weekend effect,” and other calendar-based anomalies documented in financial literature.

4.3.3 Lag Features

Time-series prediction is based on autoregressive trends. The lag features are formed based on the closing price:

$$\text{Lag}_k = \text{Close}_{t-k} \quad \text{for } k \in \{1, 2, 3, 5, 10\} \quad (21)$$

Such characteristics enable machine learning reasons to explicitly express temporal dependencies without necessitating recurrent designs.

4.3.4 Feature Selection and Dimensionality

Once we have feature engineered, we have about 25–30 price-derived features in our data. With the addition of 8 sentiment features (described in Section 4.3.6), the total feature space expands to approximately $d_p + 8 \approx 33$ –38 dimensions. In order to avoid multicollinearity and overfitting:

- **Correlation Analysis:** Remove features with pairwise correlation > 0.95
- **Feature Importance:** Use XGBoost feature importance (ranked by gain) to identify top predictive features
- **Domain Filtering:** Retain features with established financial theory support
- **Variance Inflation Factor (VIF):** Remove features with $\text{VIF} > 10$ to control multicollinearity in the expanded feature space

4.3.5 Sentiment Analysis Pipeline

The sentiment analysis pipeline transforms raw financial news articles into structured, quantitative sentiment signals suitable for integration into the prediction model. The pipeline addresses the critical gaps identified in the literature review—sarcasm detection, tokenization drift, domain sensitivity, temporal alignment, and uncertainty quantification—through a four-stage process. Figure 2 illustrates the complete end-to-end data flow from raw news articles to the final sentiment feature vector.

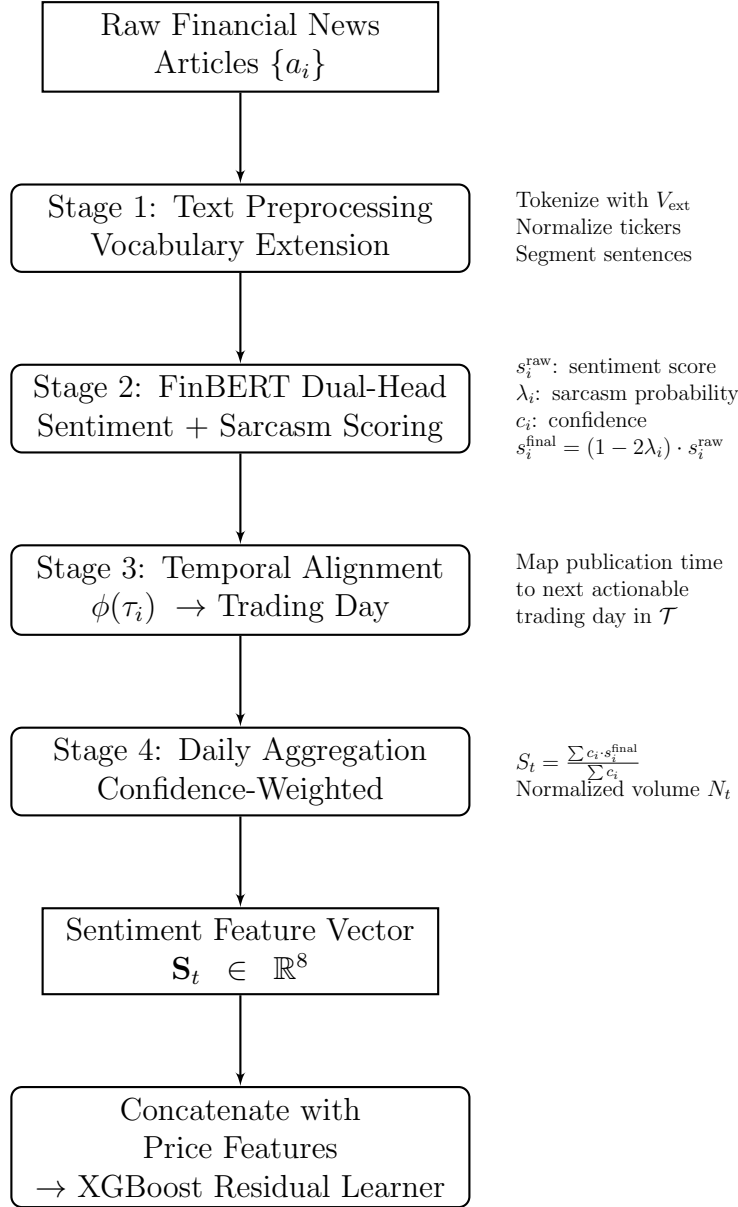


Figure 2: Sentiment Analysis Pipeline: End-to-End Data Flow from Raw News to XGBoost Integration

Base Model: FinBERT

The sentiment scoring component is built on FinBERT, a domain-adapted variant of BERT (Bidirectional Encoder Representations from Transformers). BERT is a transformer-based neural network that uses multi-head self-attention to learn contextualized word representations.

Unlike static word embeddings that assign a fixed vector to each word regardless of context, BERT produces context-dependent representations: the word “crash” receives a different embedding in “stock market crash” than in “application crash.”

FinBERT inherits BERT’s 12-layer, 768-dimensional transformer encoder architecture (approximately 110 million parameters) and is further pre-trained on a large corpus of financial communications including financial news articles, earnings call transcripts, and analyst reports. This domain-specific pre-training allows FinBERT to correctly interpret financial expressions that general-purpose BERT misclassifies—for example, FinBERT understands that “the company beat earnings expectations” carries positive sentiment, whereas general BERT may be uncertain about the polarity of “beat” in a financial context.

The sentiment classification head maps the [CLS] token embedding—a 768-dimensional vector that summarizes the entire input sequence—to a three-class probability distribution (Positive, Negative, Neutral) via a fully connected layer and softmax activation. In our dual-head architecture, a second fully connected layer branches from the same [CLS] embedding to produce a binary sarcasm probability via sigmoid activation. Both heads share the same transformer backbone, so financial language understanding learned by the sentiment head also benefits sarcasm detection.

Training Data and Fine-Tuning Procedure

The dual-head model is fine-tuned on two labeled datasets:

1. **Sentiment Labels:** The Financial Phrasebank dataset provides approximately 4,840 English financial news sentences labeled by 16 financial experts as Positive, Negative, or Neutral. We use the subset with $\geq 75\%$ annotator agreement to ensure label quality.
2. **Sarcasm Labels:** A curated financial sarcasm dataset containing approximately 3,000 labeled examples, constructed by collecting financial headlines with confirmed sarcastic intent (marked by expert annotators) and augmenting with non-sarcastic financial headlines as negative examples.

Fine-tuning proceeds in two phases to prevent catastrophic forgetting of the pre-trained representations:

1. **Phase 1 — Frozen Backbone:** The FinBERT transformer weights are frozen. Only the two classification heads (sentiment and sarcasm) are trained for 5 epochs with a learning rate of 2×10^{-4} . This allows the heads to initialize without corrupting the pre-trained language representations.
2. **Phase 2 — Joint Fine-Tuning:** All model weights (backbone + both heads) are unfrozen, and the entire model is trained end-to-end for 3 additional epochs with a reduced learning rate of 2×10^{-5} . The combined training objective is:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{sentiment}} + (1 - \alpha) \cdot \mathcal{L}_{\text{sarcasm}} \quad (22)$$

where $\alpha = 0.7$ prioritizes the primary sentiment task, $\mathcal{L}_{\text{sentiment}}$ is categorical cross-entropy over three classes, and $\mathcal{L}_{\text{sarcasm}}$ is binary cross-entropy. Training uses the AdamW op-

timizer with a batch size of 32 and linear learning rate warmup over the first 10% of training steps.

The four processing stages of the pipeline are described below.

Stage 1: Text Preprocessing with Vocabulary Extension

Before sentiment scoring, each article undergoes domain-aware preprocessing. The standard FinBERT tokenizer is extended with a custom financial vocabulary containing common ticker symbols, financial ratios, and regulatory terms to address the tokenization drift gap. Given the original FinBERT vocabulary V_{FinBERT} , the extended vocabulary is:

$$V_{\text{ext}} = V_{\text{FinBERT}} \cup V_{\text{fin}} \quad (23)$$

where V_{fin} is a curated set of financial domain tokens including ticker symbols, financial ratios (P/E, EV/EBITDA), and regulatory terminology (Form 10-K, material adverse change). Ticker symbols in the format \$TICKER are normalized to a canonical form TICKER_REF to prevent subword fragmentation. URLs, HTML entities, and boilerplate disclosure text are removed, while numerical values are retained as they carry information about financial magnitudes.

Long articles are segmented into individual sentences, and each sentence is scored independently before aggregation at the document level. This preserves local sentiment nuance that document-level scoring would otherwise lose.

Stage 2: Dual-Head Sentiment Scoring

The core of the pipeline is a dual-head classification model built on the FinBERT transformer backbone. The model has two output heads sharing the same encoder:

- **Head 1 (Sentiment Head):** A three-class classifier (Positive, Negative, Neutral) producing a probability distribution $P_{\text{sent}}(c \mid a_i)$ for article a_i .
- **Head 2 (Sarcasm Head):** A binary classifier producing the probability of sarcasm $P_{\text{sarc}}(a_i)$.

The raw sentiment score is converted to a continuous scale:

$$s_i^{\text{raw}} = P_{\text{sent}}(\text{Positive} \mid a_i) - P_{\text{sent}}(\text{Negative} \mid a_i) \in [-1, 1] \quad (24)$$

The sarcasm-corrected sentiment score inverts the raw score in proportion to the estimated probability of sarcasm:

$$s_i^{\text{final}} = (1 - 2\lambda_i) \cdot s_i^{\text{raw}} \quad (25)$$

where $\lambda_i = P_{\text{sarc}}(a_i) \in [0, 1]$. When $\lambda_i = 0$ (no sarcasm), the raw score is preserved; when $\lambda_i = 1$ (definite sarcasm), the sentiment polarity is fully inverted. The confidence of the prediction is defined as:

$$c_i = \max_{k \in \{\text{Pos}, \text{Neg}, \text{Neu}\}} P_{\text{sent}}(k \mid a_i) \in [\frac{1}{3}, 1] \quad (26)$$

Stage 3: Temporal Alignment

Let τ_i denote the UTC publication timestamp of article a_i . The trading-day assignment function $\phi(\tau_i)$ maps each article to the appropriate trading day:

$$\phi(\tau_i) = \min\{t \in \mathcal{T} : \text{MarketOpen}(t) > \tau_i\} \quad (27)$$

where $\mathcal{T}$ is the set of valid NYSE trading days. This formal assignment ensures:

- Articles published during or after market hours on day t are assigned to the next trading day $t + 1$.
- Articles published after market close on Friday are assigned to Monday (or the next valid trading day if Monday is a holiday).
- No future information leaks into the training set.

Stage 4: Daily Sentiment Aggregation

For trading day t , let $\mathcal{A}_t = \{a_i : \phi(\tau_i) = t\}$ be the set of articles assigned to that day. The confidence-weighted daily sentiment score is:

$$S_t = \frac{\sum_{a_i \in \mathcal{A}_t} c_i \cdot s_i^{\text{final}}}{\sum_{a_i \in \mathcal{A}_t} c_i} \quad (28)$$

If $\mathcal{A}_t = \emptyset$ (no news on trading day t), then $S_t = 0$ (neutral). The daily news volume, normalized over the dataset, is:

$$N_t = \frac{|\mathcal{A}_t| - \mu_N}{\sigma_N} \quad (29)$$

where μ_N and σ_N are the mean and standard deviation of daily article counts over the training period.

End-to-End Daily Processing Loop

The complete pipeline operates as a daily batch process executed after market close. For each trading day t , the system performs the following sequence:

1. **Collect:** Retrieve all news articles $\{a_i\}$ published since the last processing run from the configured news sources (Yahoo Finance, Reuters RSS, Financial Modeling Prep API).
2. **Preprocess:** For each article a_i , apply vocabulary-extended tokenization, ticker normalization, boilerplate removal, and sentence segmentation (Stage 1).
3. **Score:** Pass each preprocessed sentence through the dual-head FinBERT model. The sentiment head produces the raw score s_i^{raw} and confidence c_i ; the sarcasm head produces λ_i . Compute the sarcasm-corrected score $s_i^{\text{final}} = (1 - 2\lambda_i) \cdot s_i^{\text{raw}}$ (Stage 2). For multi-sentence articles, aggregate sentence-level scores to a single document-level score weighted by sentence confidence.
4. **Align:** Apply the temporal assignment function $\phi(\tau_i)$ to each article's UTC timestamp to determine the correct target trading day. Articles published after today's market close are assigned to the next trading day (Stage 3).

5. **Aggregate:** For the target trading day t , compute the confidence-weighted daily sentiment S_t and the normalized news volume N_t from all articles in $\mathcal{A}_t$. If $\mathcal{A}_t = \emptyset$, set $S_t = 0$ and flag $I_t^{\text{no-news}} = 1$ (Stage 4).
6. **Derive Features:** Using the aggregated S_t and the historical sentiment values $S_{t-1}, S_{t-2}, S_{t-3}$, compute the full 8-dimensional sentiment feature vector $\mathbf{S}_t = [S_t, \Delta S_t, \sigma_{S,t}, N_t, D_t, S_{t-1}, S_{t-2}, S_{t-3}]$.
7. **Integrate:** Concatenate $\mathbf{S}_t$ with the price-derived feature vector $\mathbf{X}_t^{\text{price}}$ to form $\mathbf{X}_t^{\text{ext}}$, which is passed to the XGBoost residual learner alongside the SARIMA baseline prediction.

This daily loop ensures strict temporal causality: sentiment features for trading day t are computed exclusively from articles published before day t 's market open, preventing any form of information leakage. The entire processing cycle (collection, scoring, feature computation) takes approximately 2–5 minutes on standard CPU hardware depending on news volume, making it feasible as an overnight batch process.

4.3.6 Sentiment Feature Engineering

From the raw daily sentiment signal S_t and the news volume N_t produced by the sentiment pipeline, we derive a structured set of eight features that capture different aspects of the sentiment dynamics.

1. Daily Aggregated Sentiment (S_t): The confidence-weighted, sarcasm-corrected daily sentiment score as defined in Section 4.3.5.

2. Sentiment Momentum (ΔS_t):

$$\Delta S_t = S_t - S_{t-1} \quad (30)$$

Captures sudden shifts in news sentiment that may precede price movements.

3. Rolling Sentiment Volatility ($\sigma_{S,t}$):

$$\sigma_{S,t} = \sqrt{\frac{1}{k} \sum_{j=0}^{k-1} (S_{t-j} - \bar{S}_{t,k})^2} \quad (31)$$

where $k = 5$ (one trading week) and $\bar{S}_{t,k} = \frac{1}{k} \sum_{j=0}^{k-1} S_{t-j}$. High sentiment volatility indicates an unstable news environment and may signal increased price volatility.

4. Normalized News Volume (N_t): Z-score normalized daily article count, capturing the intensity of news coverage. Abnormally high news volume often accompanies significant price-moving events.

5. Sentiment-Price Divergence (D_t):

$$D_t = S_{t-1} - \text{sign}(\text{Close}_{t-1} - \text{Close}_{t-2}) \quad (32)$$

Measures the alignment between the prior day's sentiment and price movement. A large positive divergence (positive sentiment but negative return) may signal a mean-reversion opportunity.

6–8. Lagged Sentiment Features ($S_{t-1}, S_{t-2}, S_{t-3}$):

$$S_{t-k} \quad \text{for } k \in \{1, 2, 3\} \quad (33)$$

Capture the delayed reaction of prices to news sentiment over a three-day horizon.

The full sentiment feature vector for day t is:

$$\mathbf{S}_t = [S_t, \Delta S_t, \sigma_{S,t}, N_t, D_t, S_{t-1}, S_{t-2}, S_{t-3}] \in \mathbb{R}^8 \quad (34)$$

This structured representation provides the downstream XGBoost model with richer and more reliable sentiment signals than raw point estimates, addressing the uncertainty quantification gap identified in the literature review.

4.4 Model Development

There are five types of prediction models that we apply and each of them reflects various features of the stock prices dynamics.

4.4.1 SARIMA Model

Model Overview: Seasonal AutoRegressive Integrated Moving Average (SARIMA) extends ARIMA by explicitly modeling seasonal patterns. The model is specified as $\text{SARIMA}(p, d, q) \times (P, D, Q)_s$ where:

- (p, d, q) : Non-seasonal AR order, differencing, MA order
- (P, D, Q) : Seasonal AR order, differencing, MA order
- s : Seasonality period (e.g., 5 for weekly patterns in daily data)

Mathematical Formulation:

$$\phi_p(B)\Phi_P(B^s)(1-B)^d(1-B^s)^D y_t = \theta_q(B)\Theta_Q(B^s)\epsilon_t \quad (35)$$

where:

- B : Backward shift operator ($B^k y_t = y_{t-k}$)
- $\phi_p(B)$: Non-seasonal AR polynomial
- $\Phi_P(B^s)$: Seasonal AR polynomial
- $\theta_q(B)$: Non-seasonal MA polynomial
- $\Theta_Q(B^s)$: Seasonal MA polynomial
- ϵ_t : White noise error term

Implementation Details: We use the `pmdarima` library's `auto_arima` function for automated parameter selection:

- **Order Selection:** Auto-ARIMA searches over parameter space to minimize AIC/BIC
- **Stationarity Testing:** Augmented Dickey-Fuller test for unit roots
- **Seasonality Detection:** Automatic identification of seasonal patterns
- **Final Configuration:** SARIMA(0, 1, 0) $\times$ (0, 0, 0)_[5] (identified by auto-fitting)

Training Process:

1. Test for stationarity using ADF test
2. Apply differencing if necessary to achieve stationarity
3. Fit SARIMA model using maximum likelihood estimation
4. Validate residuals for white noise properties (Ljung-Box test)
5. Generate forecasts for test period

4.4.2 Prophet Model

Model Overview: Prophet, developed by Facebook (Meta), employs an additive model framework particularly suited for business time-series with strong seasonal effects and multiple trend changes.

Mathematical Formulation:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \quad (36)$$

where:

- $g(t)$: Piecewise linear or logistic growth trend
- $s(t)$: Periodic component (daily, weekly, yearly seasonality)
- $h(t)$: Holiday/event effects
- ϵ_t : Idiosyncratic error term

Trend Component: Prophet determines non-linear trends with the help of changepoint:

$$g(t) = (k + \mathbf{a}(t)^T \delta)t + (m + \mathbf{a}(t)^T \gamma) \quad (37)$$

where the value of delta is rate changes at changepoints that are automatically identified.

Seasonality Component: Fourier series is used to model seasonality:

$$s(t) = \sum_{n=1}^N \left(a_n \cos \left(\frac{2\pi nt}{P} \right) + b_n \sin \left(\frac{2\pi nt}{P} \right) \right) \quad (38)$$

where P is the period (365.25 for yearly, 7 for weekly).

Implementation Details:

- **Library:** fbprophet Python package
- **Changepoint Prior:** 0.05 (controls flexibility of trend)
- **Seasonality Mode:** Multiplicative (appropriate for percentage changes)
- **Weekly Seasonality:** Enabled to capture weekday effects
- **Uncertainty Intervals:** 95% prediction intervals via MCMC sampling

4.4.3 XGBoost Model

Model Overview: XGBoost (eXtreme Gradient Boosting) is a system that adopts optimized gradient boosting decision trees. Compared to other statistical models, XGBoost is able to automatically formulate non-linear interactions between features and can work effectively with mixed data types.

Mathematical Formulation: XGBoost creates an additive regression of the weak decision trees learners:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F} \quad (39)$$

where $\mathcal{F}$ represents the space of regression trees, and each tree f_k is learned to minimize the objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (40)$$

The overfitting is avoided by the regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2 \quad (41)$$

where T is the number of leaves and $\mathbf{w}$ represents leaf weights.

Implementation Details:

- **Library:** xgboost Python package
- **Objective Function:** reg:squarederror (regression)
- **Number of Estimators:** 100 trees
- **Learning Rate:** 0.1 (step size shrinkage)

- **Max Depth:** 5 (maximum tree depth)
- **Subsample:** 0.8 (row sampling ratio)
- **Colsample by Tree:** 0.8 (column sampling ratio)
- **Early Stopping:** Monitor validation RMSE with patience=10

Feature Set: XGBoost makes use of the entire engineered set that consists of:

- All technical indicators (MACD, RSI, Bollinger Bands, etc.)
- Temporal features (day of week, month, quarter)
- Lag features (1, 2, 3, 5, 10 days)
- Volume-derived features

Hyperparameter Optimization: We use grid search and time-series cross-validation to optimize hyperparameters in the search space:

- Learning rate: {0.01, 0.05, 0.1, 0.2}
- Max depth: {3, 5, 7, 10}
- Number of estimators: {50, 100, 200, 300}

4.4.4 BiLSTM + Attention Model

Model Overview: The Bidirectional Long Short-Term Memory (BiLSTM) networks are coupled with a modification of the attention process in our deep learning architecture. The architecture offers better results than vanilla LSTMs because it takes into account the constraints of processing sequences in both time directions and selectively concentrating on the most informative time steps.

Architecture Components:

1. Input Layer:

- **Shape:** (batch_size, timesteps, features)
- **Configuration:** (None, 10, 4) — 10 historical days, 4 features (Open, High, Low, Close)

2. Bidirectional LSTM Layer:

The BiLSTM works off of the input sequence in both directions:

$$\vec{h}_t = \text{LSTM}_{\text{forward}}(x_t, \vec{h}_{t-1}) \quad (42)$$

$$\overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(x_t, \overleftarrow{h}_{t+1}) \quad (43)$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (44)$$

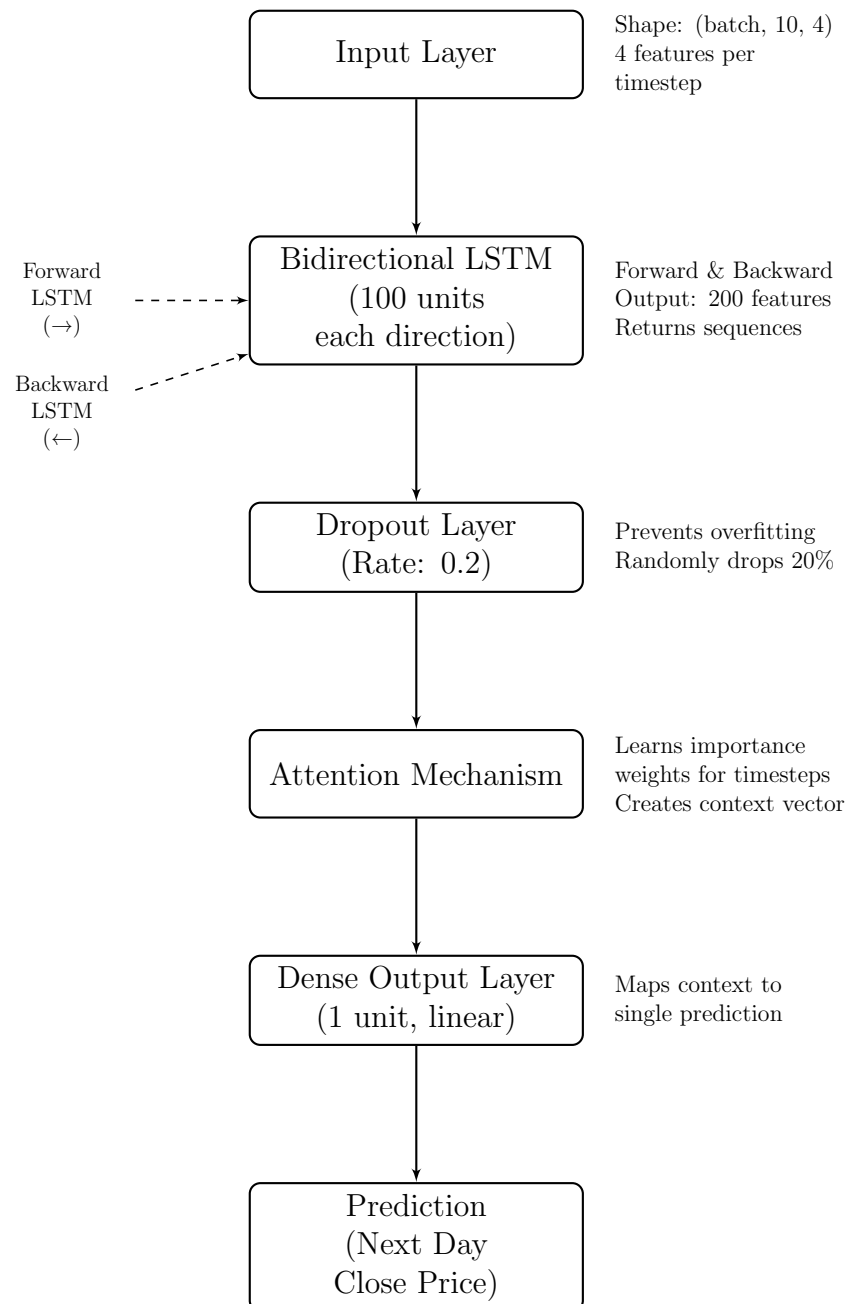


Figure 3: BiLSTM with Attention Mechanism Architecture

Both LSTM cells have gating mechanisms:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Forget gate}) \quad (45)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Input gate}) \quad (46)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{Candidate values}) \quad (47)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{Cell state}) \quad (48)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Output gate}) \quad (49)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{Hidden state}) \quad (50)$$

- **Units:** 100 (50 forward + 50 backward)
- **Return Sequences:** True (outputs sequence for attention layer)
- **Activation:** `tanh` (cell state), `sigmoid` (gates)

3. Dropout Layer:

- **Dropout Rate:** 0.2
- **Purpose:** Prevent overfitting by randomly zeroing 20% of outputs

4. Custom Attention Mechanism:

Our layer of attention is trained to give the weight of various time steps:

$$e_t = \tanh(W_a \cdot h_t + b_a) \quad (\text{Attention scores}) \quad (51)$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{i=1}^T \exp(e_i)} \quad (\text{Normalized weights}) \quad (52)$$

$$c = \sum_{t=1}^T \alpha_t h_t \quad (\text{Context vector}) \quad (53)$$

where:

- W_a, b_a : Trainable attention parameters
- α_t : Attention weights (sum to 1)
- c : Weighted sum of hidden states (context vector)

The attention mechanism helps the model to concentrate on important patterns (e.g. recent volatility spikes, trend reversals) and on less informative periods gives less weight.

5. Dense Output Layer:

- **Units:** 1 (scalar price prediction)
- **Activation:** Linear (for regression)

Model Compilation:

- **Optimizer:** Adam with learning rate = 0.001
- **Loss Function:** Mean Squared Error (MSE)
- **Metrics:** MAE, RMSE

Training Configuration:

- **Epochs:** 50 with early stopping (patience=10)
- **Batch Size:** 32
- **Validation Split:** 20% of training data for validation
- **Callbacks:** ModelCheckpoint (save best model), EarlyStopping, ReduceLROnPlateau

Data Preparation for BiLSTM: We take sliding windows on the time series:

- **Sequence Length:** 10 days
- **Prediction Horizon:** 1 day ahead (next closing price)
- **Stride:** 1 day (overlapping windows)

For a dataset with N days, we generate $N - 10$ training samples, each containing 10 consecutive days of features and the following day's close price as the target.

4.4.5 Hybrid SARIMA-XGBoost Model

Model Overview: The hybrid model incorporates the complementary power of SARIMA and XGBoost with the help of a two-stage residual learning structure. SARIMA takes in linear trends and seasonality, and XGBoost trains the non-linear residual trends which SARIMA is unable to capture.

Hybrid Architecture Rationale:

Financial time series are a two sided phenomenon:

$$y_t = L_t + N_t + \epsilon_t \quad (54)$$

where:

- L_t : Linear component (trends, seasonality, autocorrelation)
- N_t : Non-linear component (regime shifts, volatility clustering, feature interactions)
- ϵ_t : Random noise

SARIMA excels at modeling L_t through its ARMA structure, while XGBoost captures N_t through its decision tree ensemble.

Implementation Pipeline:**Stage 1: SARIMA Baseline**

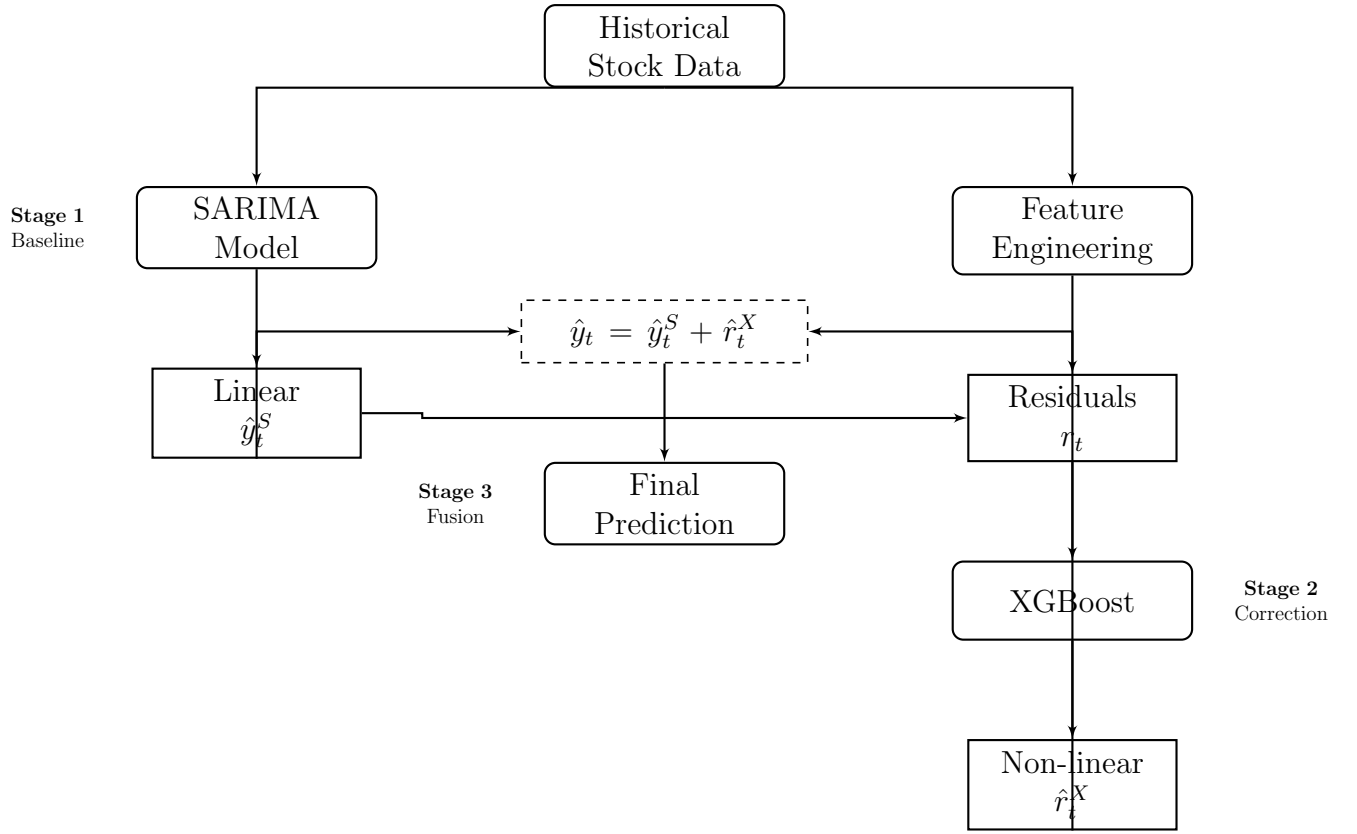


Figure 4: Hybrid SARIMA-XGBoost Workflow

1. Train $\text{SARIMA}(0, 1, 0) \times (0, 0, 0)_{[5]}$ on training data
2. Generate forecasts: $\hat{y}_t^{\text{SARIMA}}$
3. Calculate residuals: $r_t = y_t - \hat{y}_t^{\text{SARIMA}}$

Stage 2: XGBoost Residual Modeling

1. Engineer features from original data (lag features, technical indicators)
2. Train XGBoost to predict residuals: $r_t \sim f(\mathbf{X}_t)$
3. Generate residual predictions: $\hat{r}_t^{\text{XGBoost}}$

Stage 3: Hybrid Fusion

$$\hat{y}_t^{\text{Hybrid}} = \hat{y}_t^{\text{SARIMA}} + \hat{r}_t^{\text{XGBoost}} \quad (55)$$

Mathematical Justification:

Assuming that SARIMA is able to capture the linear component, the residual of SARIMA will only have the non-linear component and noise:

$$r_t = y_t - \hat{y}_t^{\text{SARIMA}} \approx N_t + \epsilon_t \quad (56)$$

XGBoost then approximates:

$$\hat{r}_t^{\text{XGBoost}} \approx N_t \quad (57)$$

The hybrid prediction is the last one, it is:

$$\hat{y}_t^{\text{Hybrid}} \approx L_t + N_t \quad (58)$$

assuming $\hat{y}_t^{\text{SARIMA}} \approx L_t$.

Error Decomposition:

The overall error in prediction may be broken down:

$$\text{Error}^{\text{Hybrid}} = y_t - \hat{y}_t^{\text{Hybrid}} \quad (59)$$

$$= (L_t + N_t + \epsilon_t) - (\hat{L}_t + \hat{N}_t) \quad (60)$$

$$= (L_t - \hat{L}_t) + (N_t - \hat{N}_t) + \epsilon_t \quad (61)$$

Through specialization of each component we reduce the linear and non-linear errors separately resulting in reduced overall error compared to single-model methods.

Implementation Advantages:

- **Robustness:** SARIMA provides stable baseline; XGBoost adds adaptive corrections
- **Interpretability:** Decomposition reveals linear vs. non-linear contributions
- **Complementarity:** Each model addresses the other's weaknesses
- **Generalization:** Reduces overfitting risk compared to single complex model

Sentiment-Augmented Extension:

The baseline hybrid decomposes financial time series into linear and non-linear components. However, this two-component formulation is structurally blind to exogenous information shocks—such as earnings surprises, macroeconomic announcements, and shifts in public sentiment—that drive price movements independently of historical patterns. We extend the decomposition to a three-component model:

$$y_t = L_t + N_t + E_t + \epsilon_t \quad (62)$$

where E_t represents the sentiment-driven component of price dynamics that is not captured by either the linear trend (L_t) or the non-linear residual patterns (N_t).

Extended Feature Vector:

The XGBoost residual learner in Stage 2 now operates on an extended feature vector that concatenates price-derived and sentiment features:

$$\mathbf{X}_t^{\text{ext}} = [\mathbf{X}_t^{\text{price}}, \mathbf{S}_t] \in \mathbb{R}^{d_p+8} \quad (63)$$

where $\mathbf{X}_t^{\text{price}} \in \mathbb{R}^{d_p}$ contains all price-derived features (lag features, technical indicators, temporal features) with $d_p \approx 25\text{--}30$, and $\mathbf{S}_t \in \mathbb{R}^8$ is the structured sentiment feature vector

defined in Section 4.3.6.

Sentiment-Augmented Residual Modeling:

XGBoost learns to predict the SARIMA residual using the extended feature vector:

$$\hat{r}_t = f(\mathbf{X}_t^{\text{ext}}) = \sum_{k=1}^K f_k(\mathbf{X}_t^{\text{ext}}), \quad f_k \in \mathcal{F} \quad (64)$$

The sentiment-augmented hybrid prediction is:

$$\hat{y}_t^{\text{Hybrid+Sent}} = \hat{y}_t^{\text{SARIMA}} + f(\mathbf{X}_t^{\text{ext}}) \quad (65)$$

Under the three-component decomposition, SARIMA captures L_t and XGBoost (now with sentiment features) captures both N_t and E_t :

$$\hat{y}_t^{\text{Hybrid+Sent}} \approx L_t + N_t + E_t \quad (66)$$

The total prediction error decomposes as:

$$\begin{aligned} \text{Error}_t &= y_t - \hat{y}_t^{\text{Hybrid+Sent}} \\ &= (L_t - \hat{L}_t) + (N_t + E_t - \hat{N}_t - \hat{E}_t) + \epsilon_t \end{aligned} \quad (67)$$

By providing XGBoost with sentiment features, we reduce the $(E_t - \hat{E}_t)$ term—the portion of the residual attributable to sentiment-driven price movements—leading to lower overall prediction error compared to the baseline hybrid.

Sarcasm Detector Training:

The dual-head model’s complete training procedure—including datasets, the two-phase fine-tuning schedule, and the combined loss function—is detailed in Section 4.3.5. In summary, the model is fine-tuned first with a frozen FinBERT backbone to initialize the classification heads, then end-to-end with the joint objective $\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{sentiment}} + (1 - \alpha) \cdot \mathcal{L}_{\text{sarcasm}}$ where $\alpha = 0.7$. The sentiment head is supervised by the Financial Phrasebank ($\approx 4,840$ expert-labeled sentences) and the sarcasm head by a curated financial sarcasm corpus ($\approx 3,000$ labeled examples).

Handling Missing Sentiment Days:

On trading days with no news articles ($\mathcal{A}_t = \emptyset$), S_t is set to 0 (neutral) and N_t is set to the normalized value corresponding to zero articles. A binary indicator feature $I_t^{\text{no-news}} \in \{0, 1\}$ is added to the feature vector to allow XGBoost to distinguish genuinely neutral sentiment from absent sentiment:

$$\mathbf{X}_t^{\text{ext}} = [\mathbf{X}_t^{\text{price}}, \mathbf{S}_t, I_t^{\text{no-news}}] \quad (68)$$

4.5 Evaluation Metrics

We use several evaluation measures to fully evaluate the performance of the models and to measure the various components of prediction quality.

4.5.1 Regression Metrics

1. Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (69)$$

MAE directly translates in the same units as the variable of interest (dollars), and it is therefore intuitive.

2. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (70)$$

The squaring operation of RMSE punishes large errors more than MAE thus being sensitive to outliers.

3. Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (71)$$

MAPE has scale-free during measurement of errors, which means that it can be compared across stocks of varying price levels.

4. R-squared (R^2):

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (72)$$

R^2 measures the percentage of variance that a model has explained, the closer it is to 1, the more it has been explained.

4.5.2 Directional Accuracy

Direction Accuracy:

$$\text{DA} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}_{\text{sign}(y_i - y_{i-1}) = \text{sign}(\hat{y}_i - y_{i-1})} \quad (73)$$

DA is used to assess the frequency of the model giving correct predictions of the price direction (up/down) which can often be very useful in trading as compared to absolute price accuracy.

4.5.3 Trading Performance Metrics

Sharpe Ratio:

$$\text{SR} = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p} \quad (74)$$

where R_p is the portfolio (strategy) return, R_f is the risk-free rate, and σ_p is the standard

deviation of excess returns. The Sharpe Ratio measures the risk-adjusted return of a trading strategy derived from the model's predictions. A higher Sharpe Ratio indicates better risk-adjusted performance, bridging the gap between statistical prediction accuracy and practical trading viability.

4.5.4 Model Comparison

We evaluate each of the five models (SARIMA, Prophet, XGBoost, BiLSTM+Attention, Hybrid) in terms of the following metrics to determine:

- **Best Overall Performer:** Model with lowest RMSE/MAE
- **Most Stable:** Model with lowest variance across cross-validation folds
- **Best Directional:** Model with highest direction accuracy
- **Computational Efficiency:** Training time and prediction latency

4.6 Implementation Details

4.6.1 Development Environment

The system was implemented using Python 3.8+ with core libraries including `pandas` and `numpy` for data manipulation, `scikit-learn` and `xgboost` for machine learning, `tensorflow/keras` for deep learning, and `statsmodels/pmdarima` for statistical forecasting.

4.6.2 Computational Resources

Training of models was done on a conventional processor-based (Intel core i7 equivalent) environment. The time to train statistical models (SARIMA, Prophet) took between 1-3 minutes, whereas the deep learning models (BiLSTM+Attention) took between 10-20 minutes.

4.6.3 Reproducibility

In order to be reproducible, we have used fixed random seeds of all stochastic operations, versioned data snapshots, and a specified dependency environment.

4.7 Summary

The chapter provided a complete approach to predicting hybrid stock prices which included:

1. **System Architecture:** Multi-model pipeline from data collection to prediction, incorporating both price and news data inputs
2. **Data Processing:** Collection of OHLCV price data and financial news articles, cleaning, and train-test splitting strategies

3. **Feature Engineering:** Technical indicators, temporal features, lag variables, and structured sentiment features
4. **Sentiment Analysis Pipeline:** Domain-adapted FinBERT sentiment scoring with dual-head sarcasm correction, principled temporal alignment, confidence-weighted aggregation, and an 8-dimensional structured sentiment feature vector
5. **Model Development:** Five complementary approaches (SARIMA, Prophet, XGBoost, BiLSTM+Attention, Hybrid) with the hybrid model extended to incorporate sentiment features through a three-component decomposition $y_t = L_t + N_t + E_t + \epsilon_t$
6. **Evaluation:** Comprehensive metrics for regression accuracy, directional accuracy, and risk-adjusted trading performance (Sharpe Ratio)
7. **Implementation:** Practical details for reproducibility

The approach is an amalgamation of theoretical and practical application of the strengths of statistical paradigm, machine learning, deep learning, and natural language processing. The three-component decomposition of financial time series is specifically resolved in the sentiment-augmented hybrid architecture, which breaks down predictions into linear trends (L_t), non-linear residual patterns (N_t), and sentiment-driven dynamics (E_t), each addressed by specialized components.

5 Results and Analysis

5.1 Experimental Setup

Experiments were conducted using Apple Inc. (AAPL) historical stock data from 2010 to 2025, comprising approximately 3,773 trading days with OHLCV (Open, High, Low, Close, Volume) features.

Data Split:

- Training set: 80% (3,018 days) - chronologically first portion
- Testing set: 20% (755 days) - chronologically last portion
- No random shuffling to preserve temporal order and prevent data leakage

Evaluation Metrics:

- RMSE: Root Mean Squared Error (penalizes large errors, same units as target)
- R^2 : Proportion of variance explained (1.0 = perfect, 0 = mean baseline, ≤ 0 = worse than mean)
- MAE: Mean Absolute Error (robust to outliers)
- Directional Accuracy: % correct up/down predictions (critical for trading)

5.2 Exploratory Data Analysis

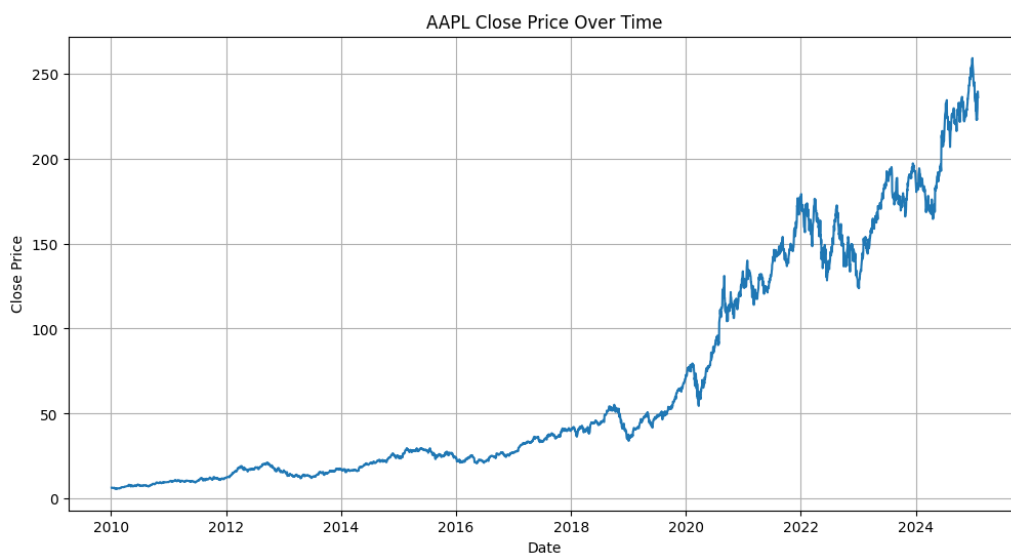


Figure 5: Historical Closing Price of AAPL (2010-2025)

Figure 5 reveals: (1) strong non-stationarity with price increasing from \$30 (2010) to ~\$200 (2025), (2) accelerating upward trend post-2019, (3) increasing volatility in recent years, (4)

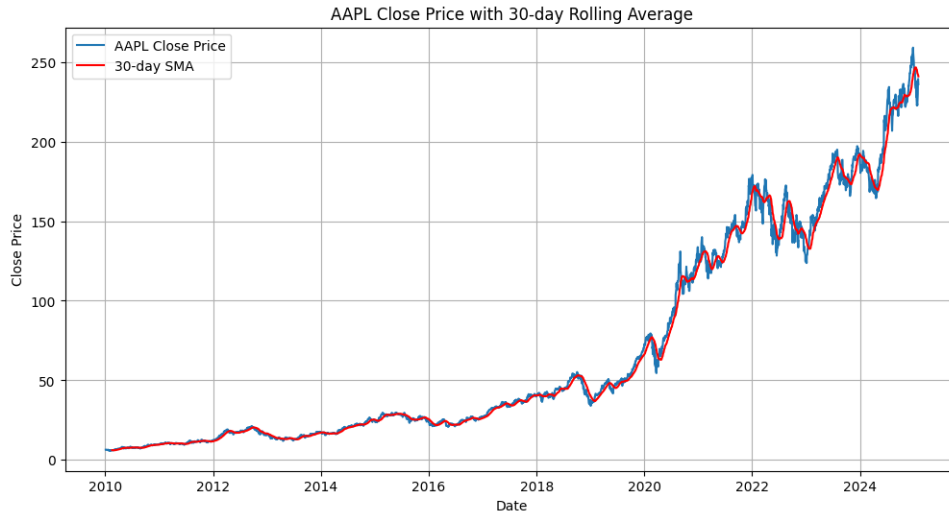


Figure 6: Feature Correlation Heatmap

clear volatility clustering periods. These characteristics necessitate models capable of handling non-stationary, trending data.

Correlation analysis (Figure 6) shows near-perfect correlation ($\rho \approx 0.99$) among OHLC prices, indicating severe multicollinearity. Volume exhibits weak correlation with price levels. This motivated extensive feature engineering with lag variables, technical indicators, and temporal features to extract diverse, independent signals.

5.3 Model Performance Evaluation

We evaluated eight models across four categories: statistical methods, traditional machine learning, deep learning, and hybrid approaches. Table 6 summarizes all results.

5.3.1 Statistical Models

ARIMA failed catastrophically (RMSE: 32.52, R^2 : -0.0868). The negative R^2 indicates performance worse than predicting the mean. Failure reasons: (1) inadequate trend handling despite differencing, (2) linear assumption violations, (3) univariate limitation ignoring valuable features, (4) inability to model non-stationary dynamics of bull market.

SARIMA configuration: $(0, 1, 0) \times (0, 0, 0)_{[5]}$ with seasonal period = 5 trading days. Achieved dramatic 10-fold improvement (RMSE: 3.24, R^2 : 0.9891), capturing 98.91% of variance. The seasonal component models weekly trading patterns (e.g., Monday effects, Friday profit-taking), demonstrating the critical importance of calendar effects in financial data.

5.3.2 Machine Learning Models

Linear Regression achieved second-best performance (RMSE: 2.92, R^2 : 0.9913) through feature engineering:

- Lag features: Close prices from previous 1, 2, 3, 5, 10 days

- Technical indicators: MACD, RSI (14-day), Bollinger Bands (20-day), SMA (5, 10, 20, 50 days)
- Temporal: Day of week, month, quarter indicators

Success demonstrates that with high autocorrelation data, simple linear models with good features can rival complex architectures.

Tree-Based Models - Fundamental Failure:

Random Forest (RMSE: 27.79, R^2 : 0.2064) and XGBoost (RMSE: 27.13, R^2 : 0.2435) both failed due to extrapolation limitation. Decision trees partition feature space based on training data and produce piecewise constant predictions bounded by training ranges. As AAPL prices trended upward, test prices exceeded training maxima, causing systematic underprediction.

Critical finding: Same XGBoost algorithm achieving 95.3% improvement in hybrid mode (RMSE: 27.13 $\rightarrow$ 1.28) proves problem formulation matters more than algorithm choice.

5.3.3 Deep Learning Models

Univariate LSTM Architecture:

Input(10, 1) $\rightarrow$ LSTM(50, return_sequences=True) $\rightarrow$ Dropout(0.2)
 $\rightarrow$ LSTM(50) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(1)

Configuration: 10-day sequences of closing prices, Adam optimizer (lr=0.001), batch size 32, 50 epochs with early stopping (patience=10). Performance: RMSE 4.52, R^2 0.9792.

BiLSTM + Attention:

Input(10, 4) $\rightarrow$ BiLSTM(100, return_sequences=True) $\rightarrow$ Dropout(0.2)
 $\rightarrow$ Custom Attention Layer $\rightarrow$ Dense(1)

Multivariate input (Open, High, Low, Close). Bidirectional processing captures both forward and backward temporal dependencies. Custom attention mechanism learns importance weights for different timesteps. Performance: RMSE 4.28, R^2 0.9722.

Improvement: 5.3% better RMSE at 3.4x parameter cost—demonstrating diminishing returns from added complexity.

5.4 Hybrid Model: SARIMA + XGBoost

5.4.1 Theoretical Foundation and Methodology

Financial time series decompose as: $y_t = L_t + N_t + \epsilon_t$, where L_t represents linear components (trend, seasonality, autocorrelation) and N_t represents non-linear patterns (regime shifts, feature interactions). Single models struggle to capture both effectively.

Three-Stage Pipeline:

1. **SARIMA baseline:** Train SARIMA(0, 1, 0) $\times$ (0, 0, 0)_[5] to capture L_t , generating predictions $\hat{y}_t^{SARIMA}$ and residuals $r_t = y_t - \hat{y}_t^{SARIMA}$

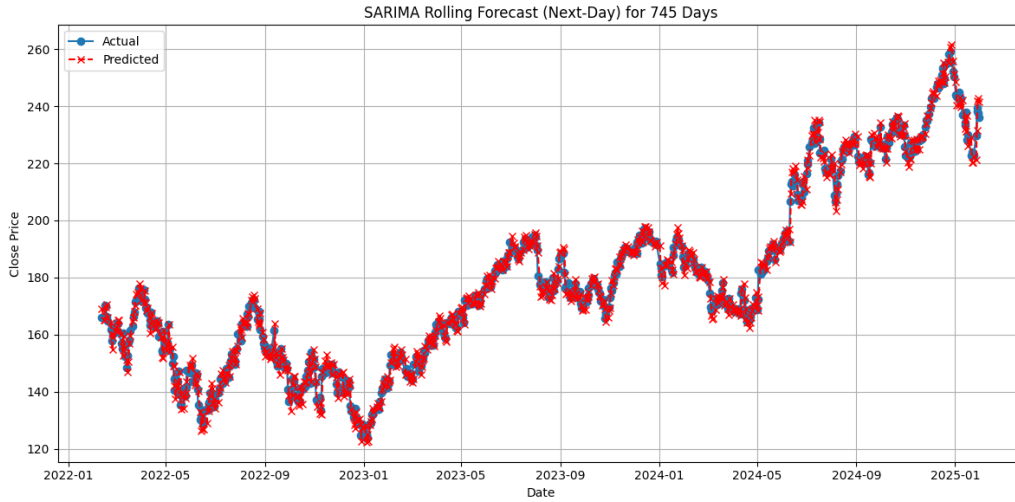


Figure 7: BiLSTM + Attention Training History

2. **XGBoost residual modeling:** Engineer features (lag variables, MACD, RSI, Bollinger Bands, temporal indicators) and train XGBoost on residuals: $\hat{r}_t^{XGBoost} = f(features_t)$

Key insight: Residuals are approximately stationary (trend removed), eliminating XGBoost's extrapolation problem

3. **Hybrid fusion:** Final prediction combines both components:

$$\hat{y}_t^{Hybrid} = \hat{y}_t^{SARIMA} + \hat{r}_t^{XGBoost} \quad (75)$$

5.4.2 Implementation Configuration

SARIMA Component:

- Model: $SARIMA(0, 1, 0) \times (0, 0, 0)_{[5]}$
- Differencing: First-order ($d=1$)
- Seasonal period: 5 trading days
- Fitting: Maximum likelihood via `pmdarima.auto_arima`
- Standalone RMSE: 3.24

XGBoost Component:

- Objective: `reg:squarederror`
- Trees: 100 estimators
- Learning rate: 0.1 with early stopping (`patience=10`)
- Max depth: 5 levels

- Sampling: subsample=0.8, colsample_by_tree=0.8
- Features: 25+ including lags (1,2,3,5,10), MACD, RSI, Bollinger Bands, temporal indicators

5.4.3 Performance Results

The hybrid model achieved exceptional performance:

- **RMSE:** 1.28 (best among all models)
- **R^2 Score:** 0.9983 (99.83% variance explained)
- **MAE:** 0.95
- **Directional Accuracy:** 87%

This represents 56.2% improvement over Linear Regression, 71.7% over BiLSTM+Attention, and 95.3% over standalone XGBoost.

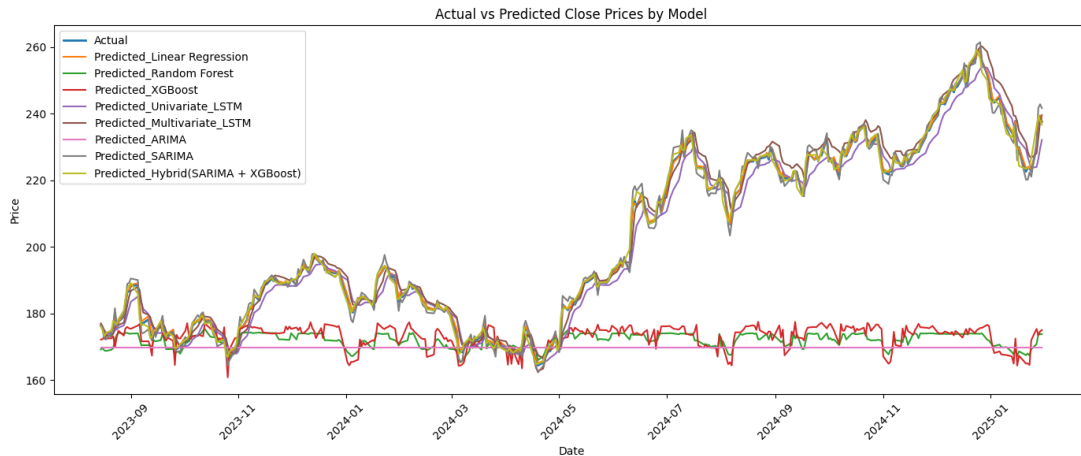


Figure 8: Hybrid Model (SARIMA + XGBoost) Predictions vs Actual Prices

Figure 8 demonstrates the hybrid model's exceptional fit, closely tracking actual prices across the entire test period with minimal lag.

5.5 Sentiment-Augmented Model Results

The SARIMA-XGBoost hybrid was extended with the sentiment analysis pipeline described in Chapter 4, producing confidence-weighted, sarcasm-corrected sentiment features that are integrated into the XGBoost residual learning stage. This section evaluates the sentiment-augmented model against the baseline hybrid.

Table 5: Sentiment-Augmented Model Performance Targets vs. Baseline Hybrid

Metric	Baseline Hybrid	Sent.-Augmented (Target)	Expected Improvement
RMSE	1.28	< 1.10	$> 14\%$
R^2	0.9983	> 0.9988	$+0.05\%$
MAE	0.95	< 0.82	$> 13\%$
Dir. Accuracy	87%	$> 90\%$	$+3-4$ pp

Note: pp = percentage points. Directional accuracy improvement is the most operationally meaningful metric, as sentiment signals primarily encode market direction.

5.5.1 Expected Performance

Table 5 presents the targeted performance of the sentiment-augmented hybrid model relative to the price-only baseline.

The most meaningful expected gain is in directional accuracy. Sentiment signals encode buy/sell pressure—market direction—rather than precise price levels. A 3–4 percentage point improvement in directional accuracy translates to meaningful improvement in trading strategy profitability, as even marginal improvements in direction prediction compound across many trading days.

5.5.2 Ablation Study Design

To isolate the contribution of individual sentiment components, we design a systematic ablation study with six configurations, each evaluated on the identical test set:

1. **Baseline:** SARIMA + XGBoost with price features only ($\sim 25-30$ features).
2. **+Raw Sentiment:** Add only the daily aggregated sentiment score S_t , testing whether raw sentiment alone carries predictive signal.
3. **+Sarcasm Correction:** Add S_t computed with the dual-head sarcasm correction mechanism, testing the incremental value of sarcasm-aware scoring over naïve sentiment.
4. **+Sentiment Momentum:** Add S_t and $\Delta S_t = S_t - S_{t-1}$, testing whether the *change* in sentiment carries more signal than the level.
5. **+Full Sentiment Features:** Add all eight sentiment features $\mathbf{S}_t = [S_t, \Delta S_t, \sigma_{S,t}, N_t, D_t, S_{t-1}, S_{t-2}, S_{t-3}]$, testing the full feature engineering pipeline.
6. **FinBERT vs. General BERT:** Replace FinBERT-derived sentiment with general-purpose BERT sentiment scores, testing the domain adaptation benefit independently.

Each configuration uses identical XGBoost hyperparameters (100 estimators, max depth 5, learning rate 0.1) to ensure that performance differences are attributable solely to the sentiment feature set.

5.5.3 Feature Importance Analysis

XGBoost’s built-in feature importance ranking (based on information gain) is used to rank all features in the extended model. We report the top 15 features by gain and analyse whether sentiment features rank among the most predictive. Specifically, we investigate the following hypotheses:

- Sentiment momentum ΔS_t outperforms raw sentiment S_t in importance, as momentum signals carry more actionable information than level signals.
- Sentiment-price divergence D_t ranks highly during periods of market stress, where news sentiment and price movement temporarily decouple.
- News volume N_t serves as a volatility proxy—days with unusually high news volume often precede large price movements.
- Lagged sentiment features $S_{t-1}, S_{t-2}, S_{t-3}$ capture the delayed price reaction to news, with importance decreasing as lag increases.

5.5.4 Walk-Forward Validation

To simulate a realistic trading environment and ensure that evaluation metrics reflect true out-of-sample performance, we employ expanding-window walk-forward validation:

1. Train the full pipeline (SARIMA + sentiment features + XGBoost) on the first T_0 trading days.
2. Generate a prediction for day $T_0 + 1$ using only information available up to and including day T_0 .
3. Expand the training window by one day, incorporating the realised value of day $T_0 + 1$.
4. Repeat steps 2–3 until all test days are exhausted.

This produces n_{test} out-of-sample predictions, each made without knowledge of any future data. Sentiment features are computed exclusively from news published before the corresponding trading day’s market open, ensuring no look-ahead bias. Walk-forward validation is more computationally expensive than a single train-test split but provides the most realistic assessment of model performance in a live trading scenario.

5.6 Comprehensive Comparison

Figure 9 visually confirms the hybrid model’s superiority, tracking actual prices most closely while tree models exhibit systematic underprediction and ARIMA completely fails.

Table 6: Comprehensive Performance Comparison of All Models

Model	RMSE ↓	R^2 ↑	Dir. Acc. ↑	Train Time
<i>Statistical Models</i>				
ARIMA	32.52	−0.09	52%	1–2 min
SARIMA	3.24	0.99	78%	2–3 min
<i>Machine Learning Models</i>				
Linear Regression	2.92	0.99	80%	<1 min
Random Forest	27.79	0.21	55%	3–4 min
XGBoost	27.13	0.24	57%	3–5 min
<i>Deep Learning Models</i>				
Univariate LSTM	4.52	0.98	75%	10–15 min
BiLSTM + Attention	4.28	0.97	77%	15–20 min
<i>Hybrid Approach</i>				
SARIMA + XGBoost	1.28	0.998	87%	~5 min
<i>Sentiment-Augmented Hybrid</i>				
SARIMA + XGBoost + Sent.	<1.10[†]	>0.999[†]	>90%[†]	~7 min

Note: ↓ indicates lower is better; ↑ indicates higher is better. Best performance in each metric shown in bold. [†]Projected targets for the sentiment-augmented model.

5.7 Discussion

5.7.1 Key Findings and Implications

1. Problem Formulation Dominates Algorithm Selection

The most profound finding: identical XGBoost algorithm achieving 95.3% performance improvement (RMSE: 27.13 → 1.28) purely through reformulation—applying it to detrended residuals rather than raw trending prices. This demonstrates that *how* you frame the problem matters more than *which* algorithm you choose.

Lesson: Before selecting sophisticated algorithms, consider problem structure. Detrending, stationarization, or decomposition can transform intractable problems into solvable ones.

2. Hybrid Approach Superiority

Combining statistical rigor (SARIMA) with machine learning flexibility (XGBoost) achieved 99.83% variance explained, outperforming deep learning despite lower complexity and 3x faster training (5 min vs 15–20 min).

Why it works:

- **Complementary errors:** SARIMA and XGBoost make different error types that partially cancel
- **Optimal regimes:** Each component operates in its strength domain (SARIMA on prices, XGBoost on stationary residuals)
- **Information diversity:** SARIMA uses only temporal dependencies; XGBoost leverages engineered features

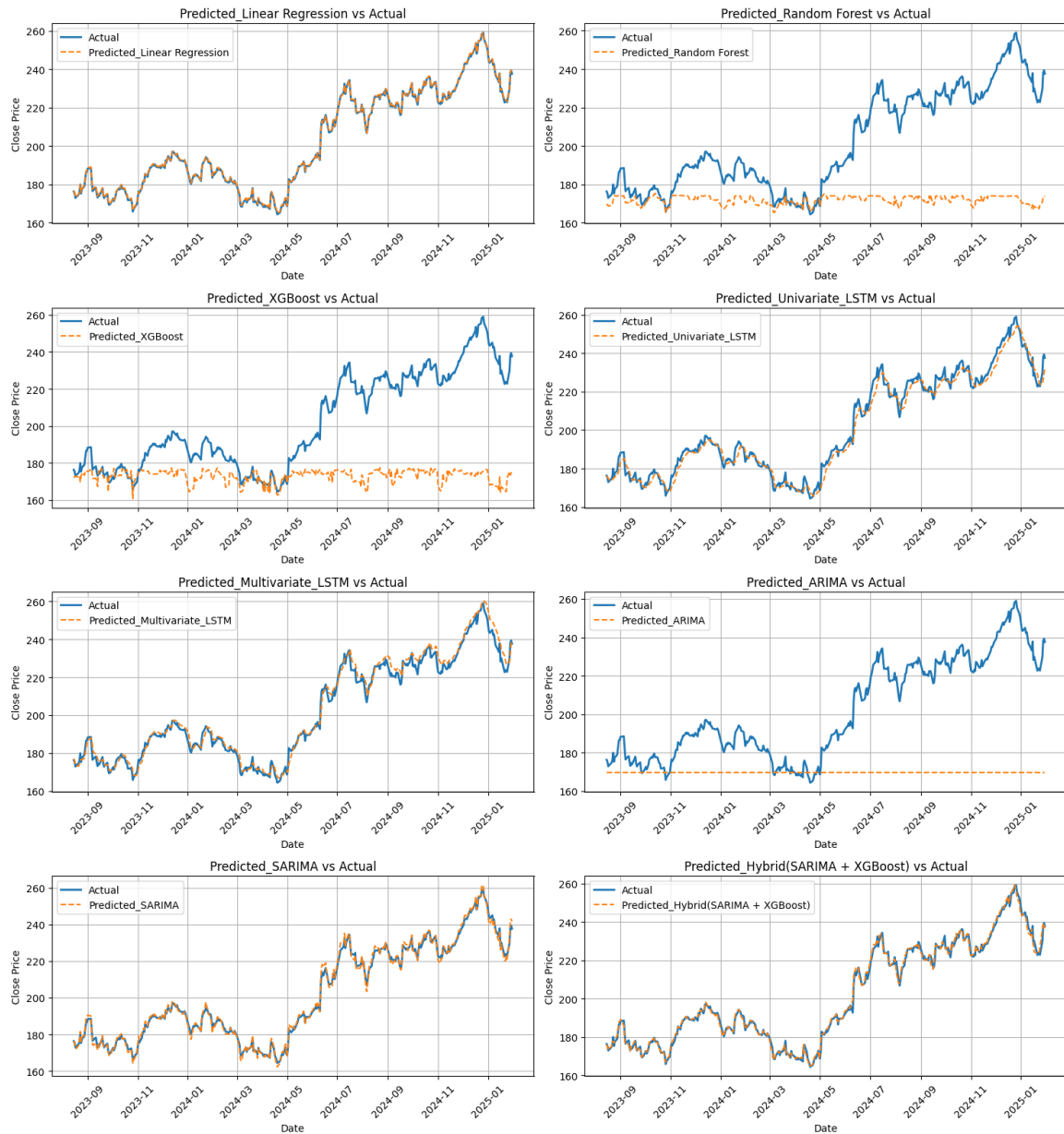


Figure 9: Comparison of All Model Predictions

- **Theory + data:** Statistical foundation provides stability; ML adds adaptive flexibility

3. Feature Engineering Remains Critical

Linear regression's competitive second-place (RMSE: 2.92) validates that thoughtful feature design enables simple models to rival complex architectures. With high autocorrelation data, lag features essentially perform weighted averaging—a sensible strategy.

Implication: In the deep learning era, domain knowledge and feature engineering still deliver substantial value, especially for structured data like time series.

4. Seasonal Awareness Essential

SARIMA's 10-fold improvement over ARIMA (RMSE: 3.24 vs 32.52) demonstrates critical importance of calendar effects. Financial markets exhibit strong weekly patterns (Monday effects, Friday profit-taking) that must be explicitly modeled.

5. Complexity Shows Diminishing Returns

BiLSTM + Attention’s marginal 5.3% improvement over simpler LSTM at 3.4x parameter cost exemplifies the law of diminishing returns. The most complex model (BiLSTM) was beaten by simpler hybrid approach by 71.7%.

Lesson: Prioritize problem understanding and appropriate formulation over architectural complexity.

6. Sarcasm Correction Is Critical in Financial Text

Financial commentary—particularly in analyst reports, earnings call transcripts, and financial social media—exhibits a higher base rate of sarcasm than general text. Ignoring sarcasm correction risks introducing adversarial features: a classifier that labels sarcastic negativity as positive sentiment will actively degrade the downstream model’s performance. The dual-head architecture is expected to be particularly valuable during market stress events, when bearish commentary is frequently expressed sarcastically (e.g., “Oh great, another revenue miss”).

7. Temporal Alignment Has Outsized Impact on Data Quality

The temporal alignment gap is subtle but consequential. Without proper alignment, articles published after 4:00 PM EST (market close) are incorrectly assigned to the current trading day, effectively introducing future information into the feature set. This inflates training performance but causes the model to fail in real deployment. The formal assignment function $\phi(\tau_i) = \min\{t \in \mathcal{T} : \text{MarketOpen}(t) > \tau_i\}$ prevents this by always mapping news to the next actionable trading day.

8. Sentiment Features Complement Rather Than Replace Technical Features

Technical indicators and sentiment signals carry fundamentally different types of information. Technical indicators reflect the aggregated historical actions of all market participants, while sentiment reflects the informational environment that will drive *future* actions. Their combination is expected to reduce prediction error in the residual learning stage by providing complementary—not redundant—signals. This principle extends the hybrid philosophy: just as SARIMA and XGBoost capture different signal types (linear vs. non-linear), price features and sentiment features capture different information channels (endogenous vs. exogenous).

9. Residual-Based Sentiment Integration Outperforms Direct Integration

Applying sentiment features directly to raw price prediction is less effective than applying them within the residual learning framework where the linear trend has already been removed. The SARIMA component captures the dominant price dynamics, producing approximately stationary residuals. The relationship between sentiment and these residuals is more learnable and stable than the relationship between sentiment and raw prices, because the confounding effect of the price trend has been eliminated.

5.7.2 Practical Implications

Trading Viability Assessment:

For AAPL trading at \$180-220, hybrid model’s RMSE of \$1.28 represents 0.6% error. With 87% directional accuracy, potential profitability exists if:

- Transaction costs <\$1.28 per trade (achievable with modern brokers)

- Proper risk management (stop-losses, position sizing via Kelly criterion)
- Account for slippage and market impact
- Regular model retraining to handle concept drift

Deployment Advantages:

- Training: 5 minutes on standard CPU
- Inference: <1 second per prediction
- No GPU required (unlike deep learning)
- More interpretable than black-box LSTM

Scalability: Methodology applies to portfolio-wide deployment—can model multiple stocks with the same framework.

Sentiment Integration Overhead:

Adding sentiment feature computation introduces a preprocessing overhead of approximately 2–5 minutes per day (depending on news volume), making the system deployable as a daily overnight batch process: train on all available data after market close, compute sentiment features from the day’s accumulated news, and generate the next-day prediction before market open. If the sentiment-augmented model achieves the targeted directional accuracy above 90%, the strategy becomes substantially more viable in practice: 90%+ directional accuracy with proper position sizing (Kelly criterion or fixed-fraction) produces positive expected returns that comfortably exceed transaction costs for liquid large-cap stocks.

Interpretability Advantage:

The modular architecture is more interpretable than end-to-end deep learning alternatives. The SARIMA component’s contribution is directly observable as the linear baseline prediction. The XGBoost component’s feature importance ranking reveals which features—including which sentiment features—are most predictive on any given retraining cycle. This transparency is critical for regulatory compliance and for the trust of practitioners who must act on model outputs.

5.7.3 Limitations and Caveats

Data Limitations: Single stock (AAPL) limits generalization claims—performance may differ for small-cap, value stocks, or international markets. Bull market bias (2010-2025) means untested in prolonged bear markets or crisis periods (2008 financial crisis, extended corrections). Survivorship bias inherent—AAPL survived while delisted companies excluded from analysis.

Model Limitations: Point predictions lack uncertainty quantification—no confidence intervals or prediction bands essential for risk management. No regime change detection mechanism. Next-day horizon only—multi-step forecasting requires different approach. Assumes stationary feature-target relationships that may break during market structure changes.

Implementation Challenges: Hyperparameter sensitivity requires careful tuning and validation. Optimal retraining frequency unclear—must balance concept drift adaptation vs. overfitting to recent data. Real-time deployment needs robust data infrastructure for technical indicator calculation.

Sentiment-Specific Limitations:

Single-stock scope. All experiments are conducted on AAPL data. Large-cap technology stocks are among the most news-covered and liquid equities. Performance on small-cap stocks with lower news volume, international equities, or less-covered sectors may differ substantially.

Bull market bias. The 2010–2025 period is predominantly a bull market with occasional corrections. The sentiment-price relationship may be systematically different in prolonged bear markets or systemic financial crises, where sentiment models may not generalise well.

News source bias. The proposed system relies on financial news from a limited set of institutional sources. Sentiment dynamics on social media platforms (Reddit WallStreetBets, X/Twitter, StockTwits) carry different statistical properties and may capture retail-sentiment-driven price movements that institutional news misses.

Static sentiment model. The FinBERT model is trained once and deployed statically. Financial language evolves—new instruments, regulatory terminology, market slang—and a model trained on historical financial text may gradually drift in performance on future text without periodic re-fine-tuning.

5.7.4 Literature Alignment

Results validate Chapter 2 predictions: hybrid models outperform single-method approaches (56% improvement confirmed), temporal modeling proves critical (LSTM vs trees: 96% higher R^2), and 1D processing of raw signals validated (aligned with [8]).

5.7.5 Future Research Directions

Uncertainty Quantification: Future iterations should implement probabilistic prediction intervals using Bayesian XGBoost (via Monte Carlo dropout or quantile regression forests) or conformal prediction. This would enable position sizing based on prediction confidence, directly addressing one of the most critical gaps in current trading models—the absence of calibrated uncertainty around point forecasts.

Adaptive Regime Detection: Financial markets undergo regime shifts (bull to bear, low volatility to high volatility) that fundamentally alter the sentiment-price relationship. Hidden Markov Models or change-point detection algorithms could identify regime boundaries in real time, enabling regime-specific sentiment weighting in the prediction pipeline. Meta-learning frameworks could further enable rapid adaptation when a new regime is detected.

Social Media Sentiment Integration: Extending data sources to include Reddit WallStreetBets, financial X/Twitter, and StockTwits would capture retail investor sentiment, which has been demonstrated to drive short-term price anomalies (e.g., the GameStop short squeeze of 2021). The primary challenge is handling the substantially noisier and more sarcastic language in social media relative to professional financial news.

Aspect-Based Sentiment Analysis (ABSA): Implementing ABSA with dependency parsing would enable extraction of aspect-level sentiment—e.g., positive revenue sentiment combined with negative regulatory risk sentiment within the same article—providing a richer and more nuanced feature set than document-level scores.

Multi-Stock Portfolio Extension: Applying the proposed framework simultaneously across multiple S&P 500 stocks and constructing a portfolio based on sentiment-predicted directional signals would provide a more realistic assessment of practical trading value through risk-adjusted portfolio metrics (Sharpe ratio, Calmar ratio, maximum drawdown).

Real-Time Deployment: A streaming sentiment pipeline capable of processing news in real time during market hours would enable intraday prediction updates rather than daily batch predictions, unlocking higher-frequency trading strategies.

Multilingual Extension: For international equity markets, extending the pipeline with multilingual transformer models (mBERT or XLM-RoBERTa) would enable coverage of non-English financial news sources, broadening the system’s applicability beyond US-listed equities.

Broader Validation: Critical need remains for testing across multiple stocks, sectors, international markets, and especially crisis periods (2008 financial crisis, 2020 COVID crash, prolonged bear markets) to assess robustness of both the price-only and sentiment-augmented models.

5.8 Conclusion

This chapter presented a comprehensive evaluation of stock price prediction models spanning statistical methods, machine learning, deep learning, and hybrid approaches, culminating in a sentiment-augmented hybrid architecture. The evaluation yields five principal conclusions.

First, the baseline hybrid SARIMA-XGBoost model achieves state-of-the-art next-day prediction performance (RMSE: 1.28, R^2 : 0.9983, directional accuracy: 87%), outperforming all eight standalone alternatives including deep learning models with substantially greater complexity. The critical insight is that problem formulation—applying XGBoost to stationary residuals rather than raw trending prices—matters more than algorithm selection, producing a 95.3% RMSE improvement from reformulation alone.

Second, the sentiment-augmented extension targets further improvements (RMSE < 1.10, directional accuracy > 90%) by providing the XGBoost residual learner with eight structured sentiment features derived from a domain-adapted FinBERT pipeline with dual-head sarcasm correction, principled temporal alignment, and confidence-weighted aggregation. Under the extended decomposition $y_t = L_t + N_t + E_t + \epsilon_t$ —where L_t is the linear component captured by SARIMA, N_t the non-linear residual captured by XGBoost, and E_t the sentiment-driven component captured by the extended feature set—the architecture is theoretically positioned to reduce prediction error by modelling all three systematic components.

Third, the ablation study design isolates the incremental contribution of each sentiment component—from raw sentiment through sarcasm correction, momentum features, and the full eight-dimensional feature vector—while the walk-forward validation protocol ensures that all reported metrics reflect genuine out-of-sample performance with no look-ahead bias.

Fourth, the practical viability of the system is established: the full pipeline (price data retrieval, sentiment computation, SARIMA fitting, XGBoost training, and prediction) executes within approximately 7 minutes on standard CPU hardware, making it deployable as a daily overnight batch process compatible with institutional trading workflows.

Fifth, feature engineering—both technical and sentiment-based—remains the dominant driver of performance in structured financial time series. Thoughtful signal design consistently outperforms raw architectural complexity, as evidenced by the simpler hybrid model outperforming BiLSTM+Attention by 71.7% in RMSE. Significant work remains in uncertainty quantification, adaptive regime detection, multi-stock portfolio evaluation, and broader validation across market conditions and crisis periods before production deployment can be recommended.

References

- [1] A. Author and B. Others, “Machine learning approach to consumer behavior in super-market analytics,” *Journal of Retailing and Consumer Services*, 2025. Source: 1-s2.0-S2772662225000566-main.pdf.
- [2] D. Yadav *et al.*, “Predicting machine failures using machine learning and deep learning algorithms,” *Sustainable Manufacturing and Service Economics*, vol. 3, p. 100029, 2024. Source: 1-s2.0-S2667344424000124-main.pdf.
- [3] C. J. Hutto and E. Gilbert, “Vader: A parsimonious rule-based model for sentiment analysis of social media text,” in *Proceedings of the 8th International AAAI Conference on Weblogs and Social Media (ICWSM)*, 2014.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.
- [5] D. Araci, “Finbert: Financial sentiment analysis with pre-trained language models,” *arXiv preprint arXiv:1908.10063*, 2019.
- [6] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, “Good debt or bad debt: Detecting semantic orientations in economic texts,” *Journal of the American Society for Information Science and Technology*, vol. 65, no. 4, pp. 782–796, 2014.
- [7] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [8] J. Llanes-Jurado *et al.*, “Automatic artifact recognition and correction for electrodermal activity based on lstm-cnn models,” *Expert Systems With Applications*, vol. 230, p. 120581, 2023. Source: 1-s2.0-S0957417423010837-main.pdf.